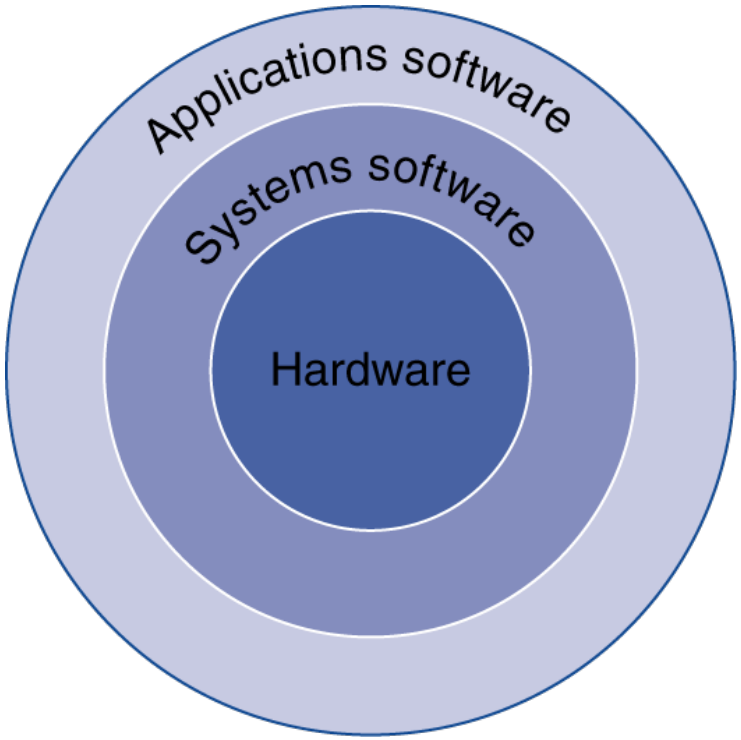
4小时学会计算机组成与结构

—上卷

- Computer Abstractions
- Performance
  - Latency, Power, Amdahl's Law, Impact Factors
- ISA
  - Design Principles, Formatting, Types and Operations, Procedure, Addressing
- Arithmetic Operations
  - Adders, Multipliers, Dividers, Floating point
- The Processor
  - Single Cycle, Datapath, Five Stages, Pipeline, Hazards, Speculation, Multi-Issue

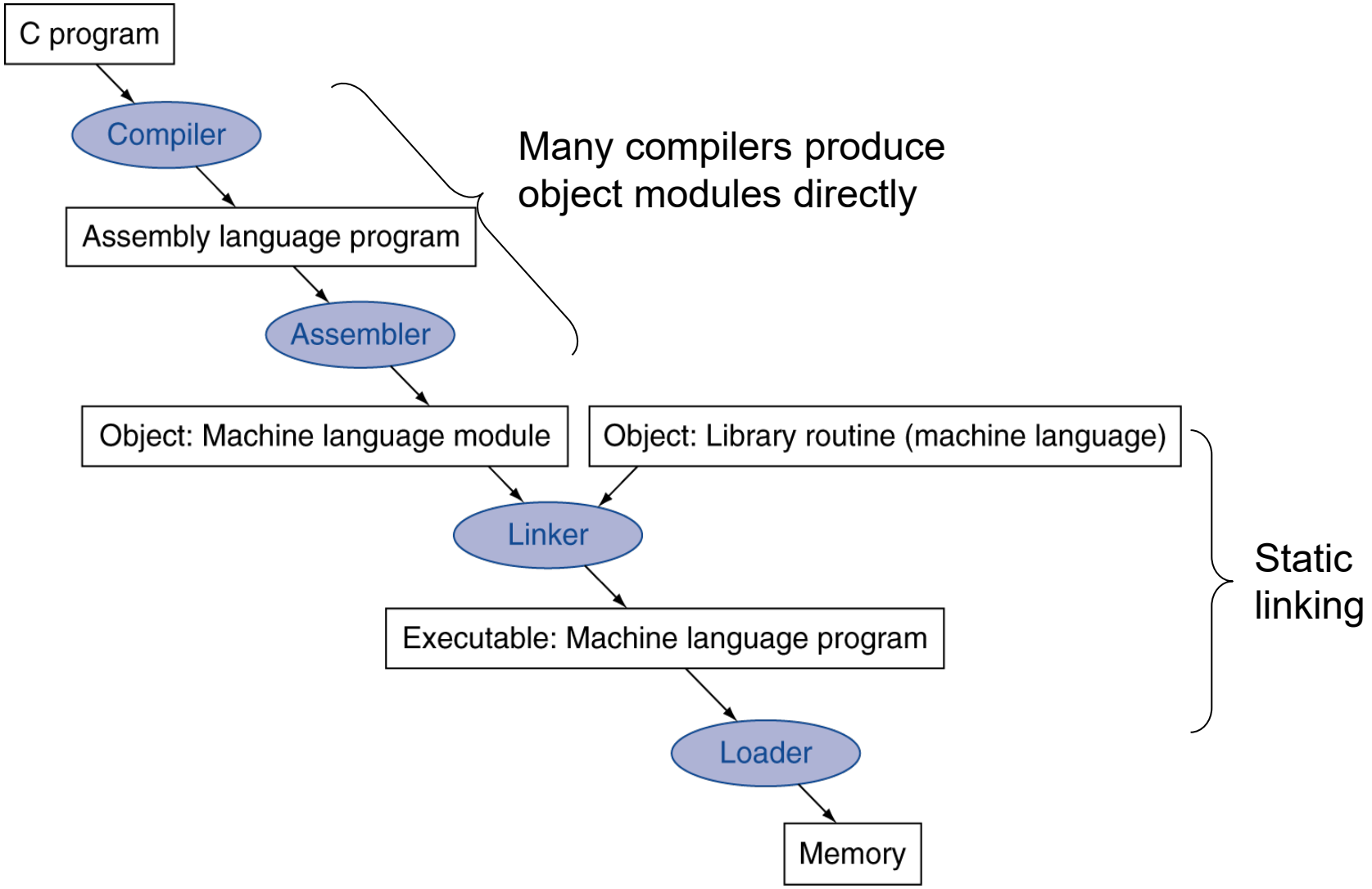
Computer Abstractions and Technology

Below Your Program



- Application software
  - Written in high-level language
- System software
  - Compiler: translates HLL code to machine code
  - Operating System: service code
    - Handling input/output
    - Managing memory and storage
    - Scheduling tasks & sharing resources
- Hardware
  - Processor, memory, I/O controllers

Translation and Startup



http://www.chuquan.me/2018/05/21/elf-introduce/

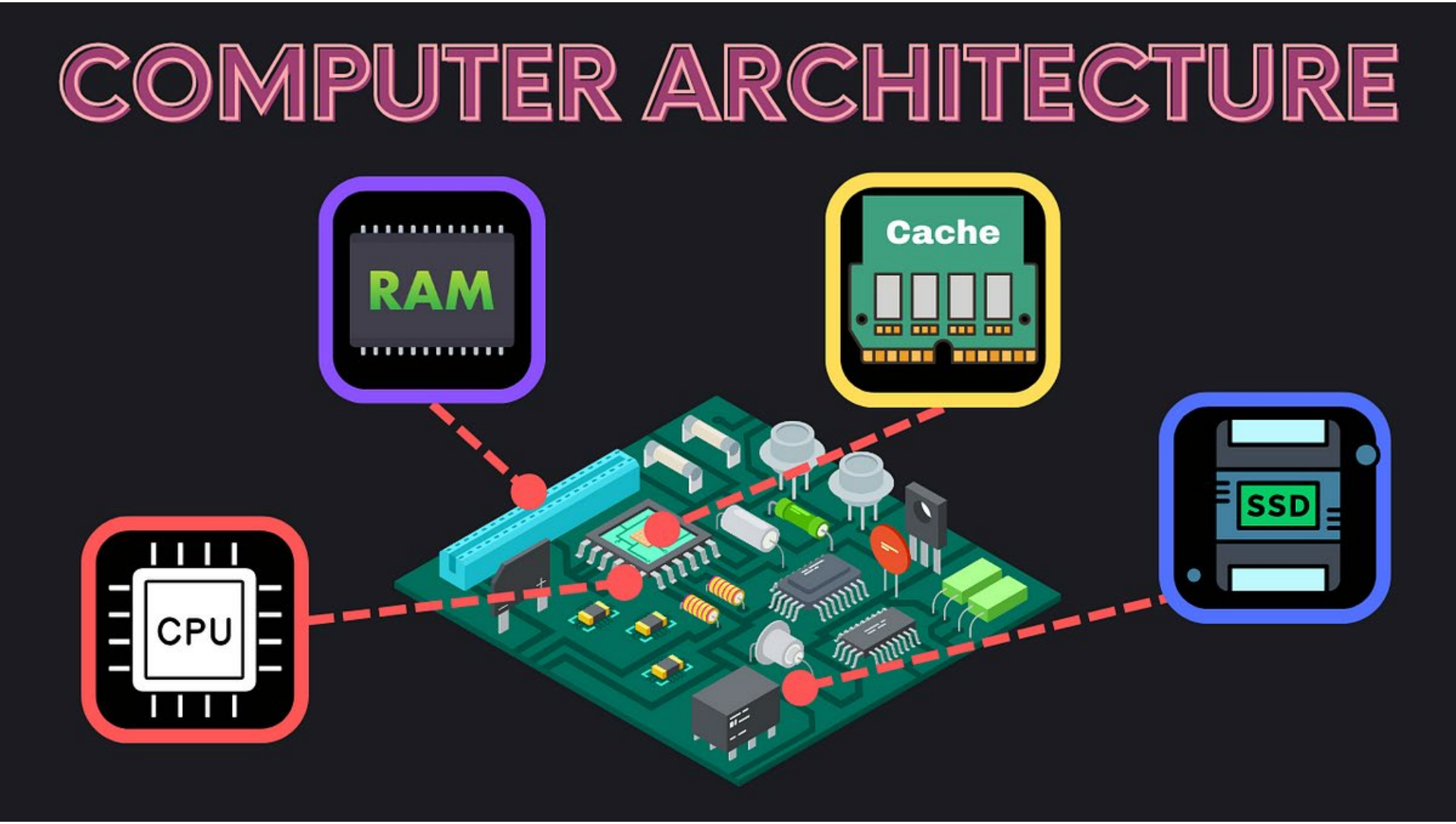
Linking Object Modules

- Produces an executable image
  1. Merges segments/object files/procedures
     - Rather than compiling/assembling whole program when a single line of a procedure is changed, procedures are compiled/assembled independently
  2. Resolve labels (determine their addresses)
  3. Patch location-dependent and external refs

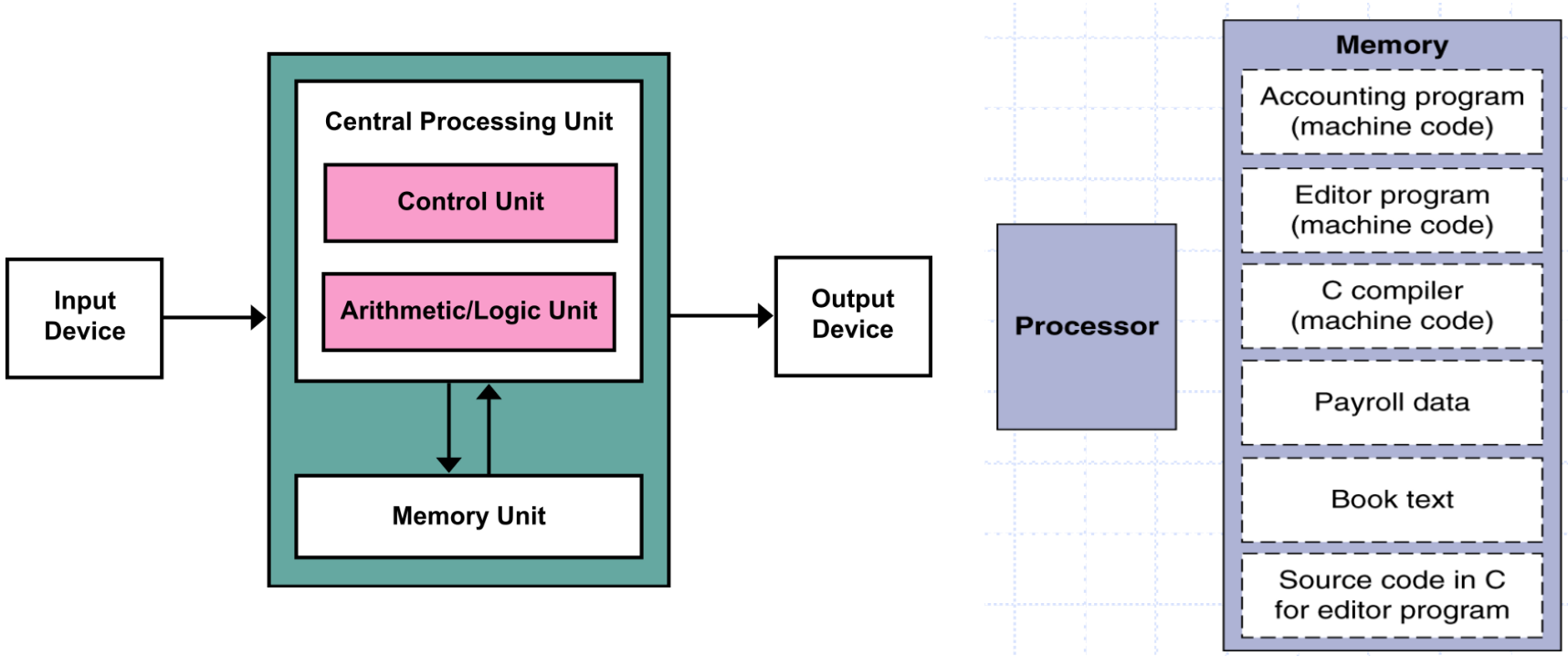
Static and Dynamic Linking

- So far discussed traditional approach to linking libraries before program executes
  - Static linking
    - If new version of a library released to fix bugs or support new hardware, statically linked program keeps using old version
    - All routines in a library are loaded, even if they are not used
- Dynamic linking: Do not link/load library routines until program is run
  - Requires procedure code to be relocatable
  - Automatically picks up new library versions
  - Dynamically linked libraries (DLLs)

Computer Architecture

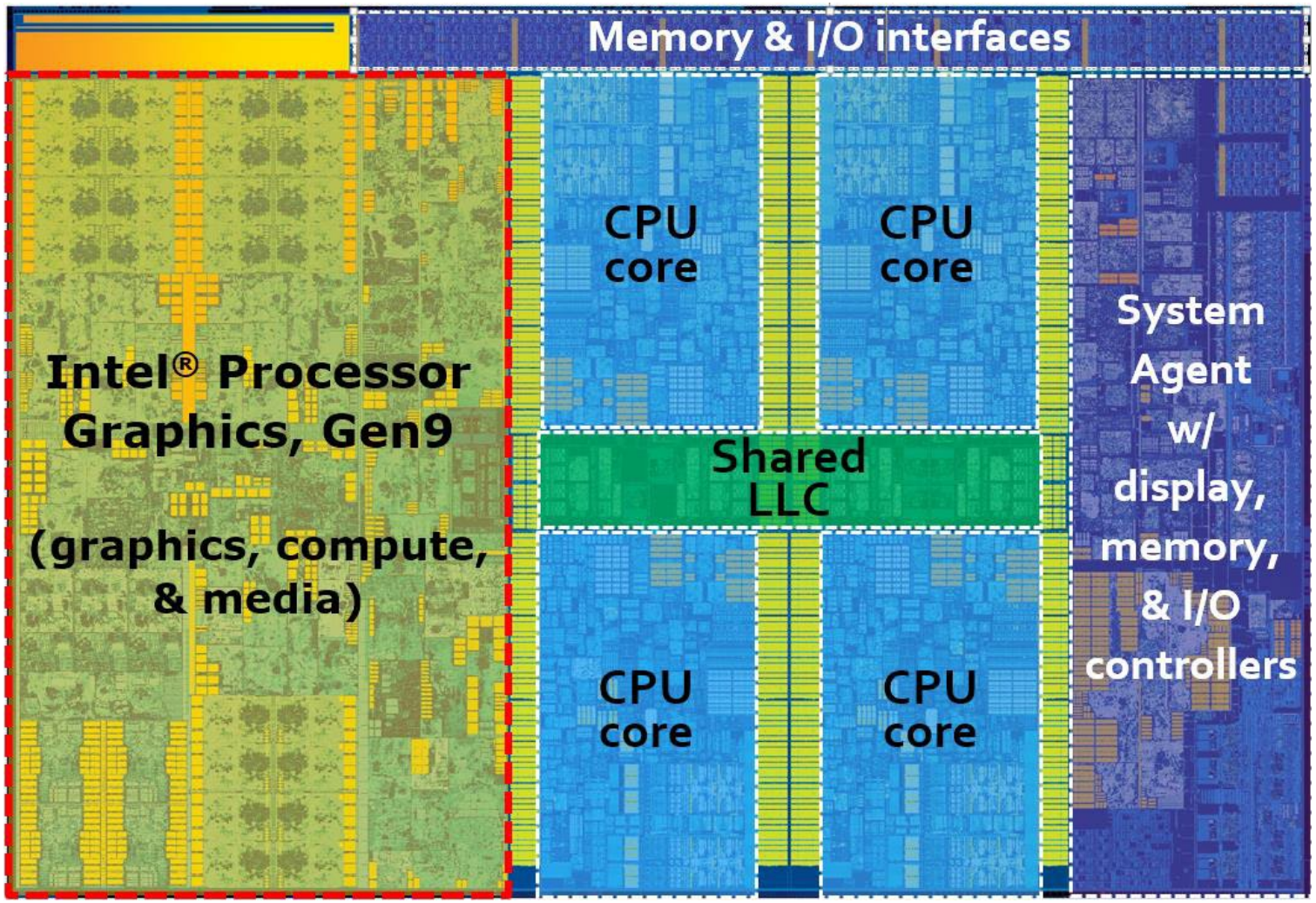


Computer Organization and The Stored-Program Concept



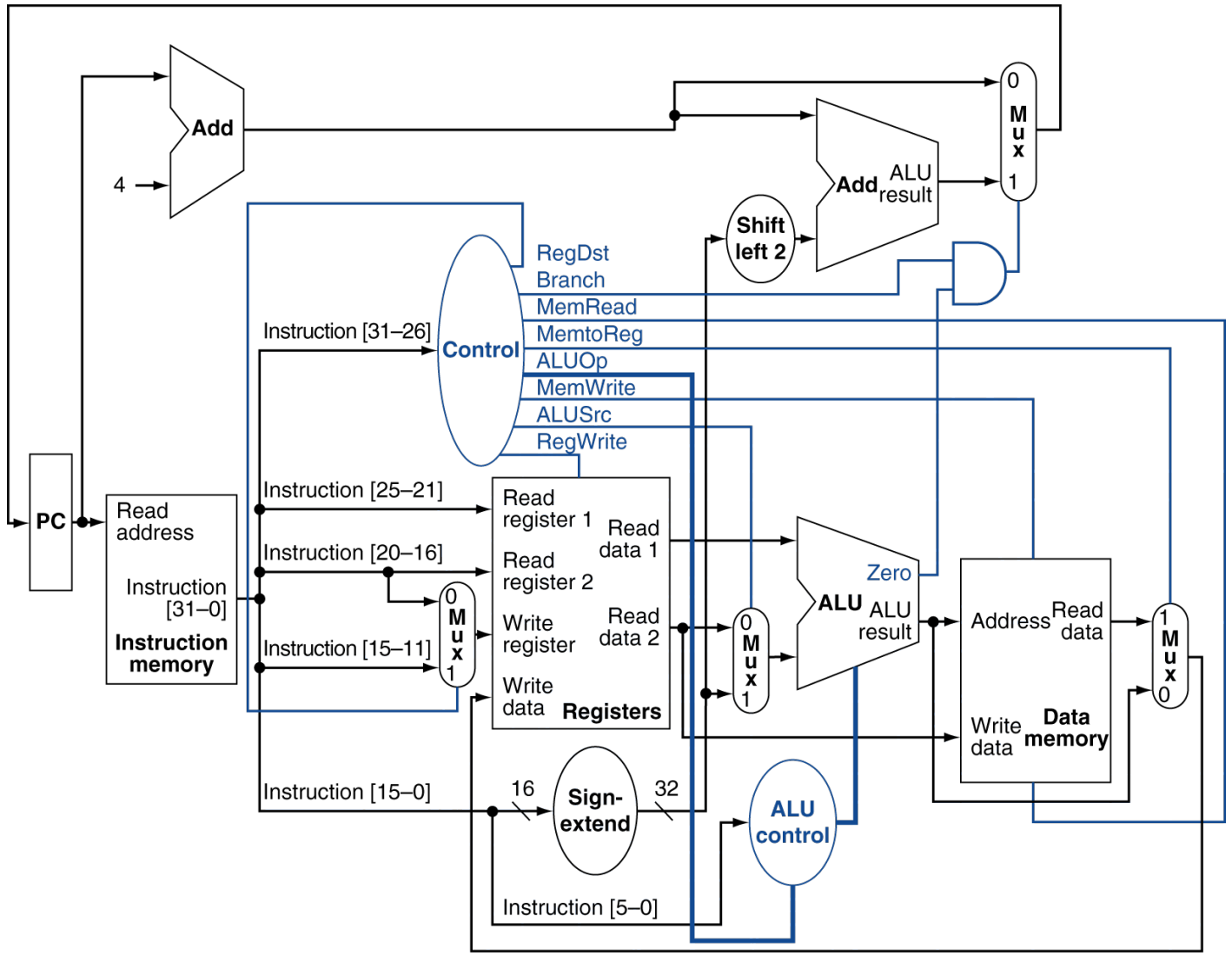
- The Stored-Program Concept
  - instructions are represented as numbers
  - programs are stored in memory just like data
- Simplify both memory HW and the SW of computers
  - programs are often exchanged as binary files
    - issue of binary compatibility and role of instruction-set architecture

Typical Computer Architectures



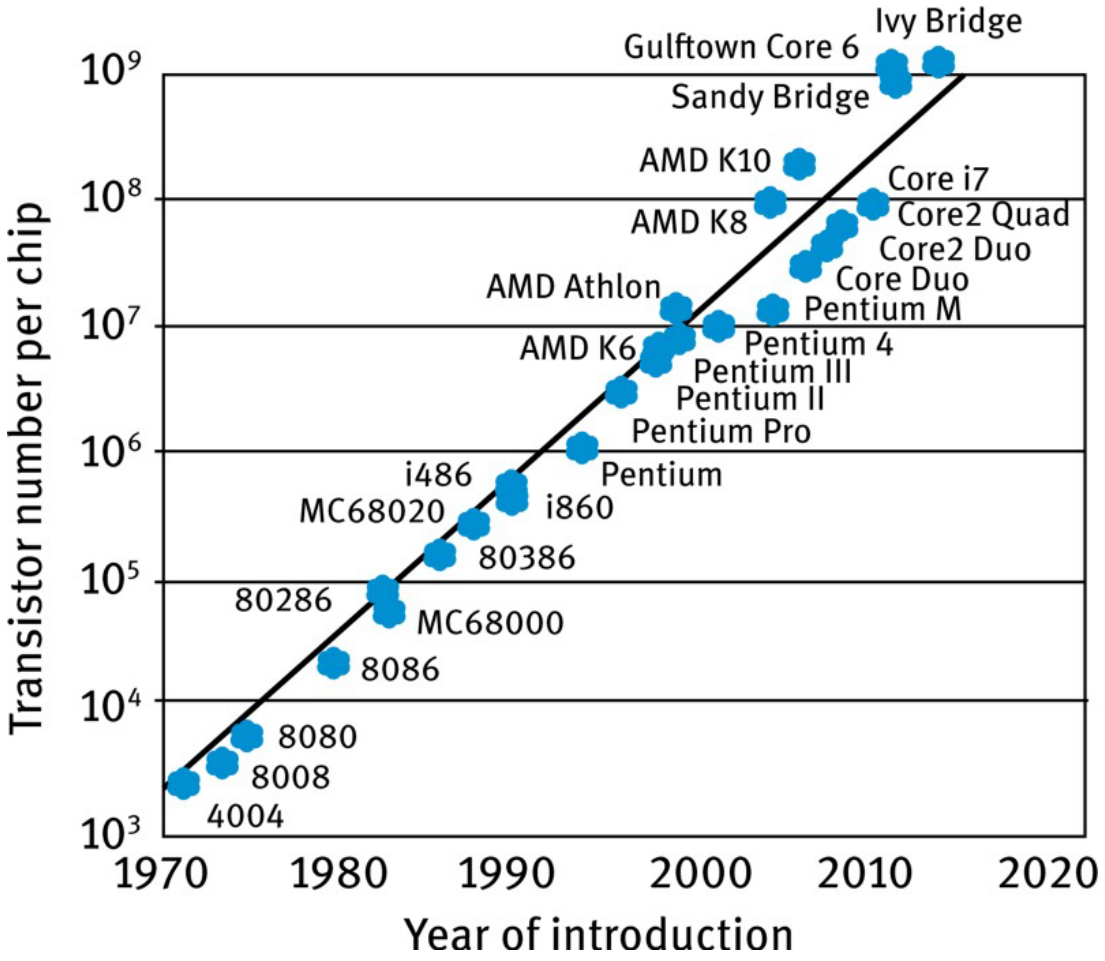
Intel Skylake Architecture

Inside the Processor

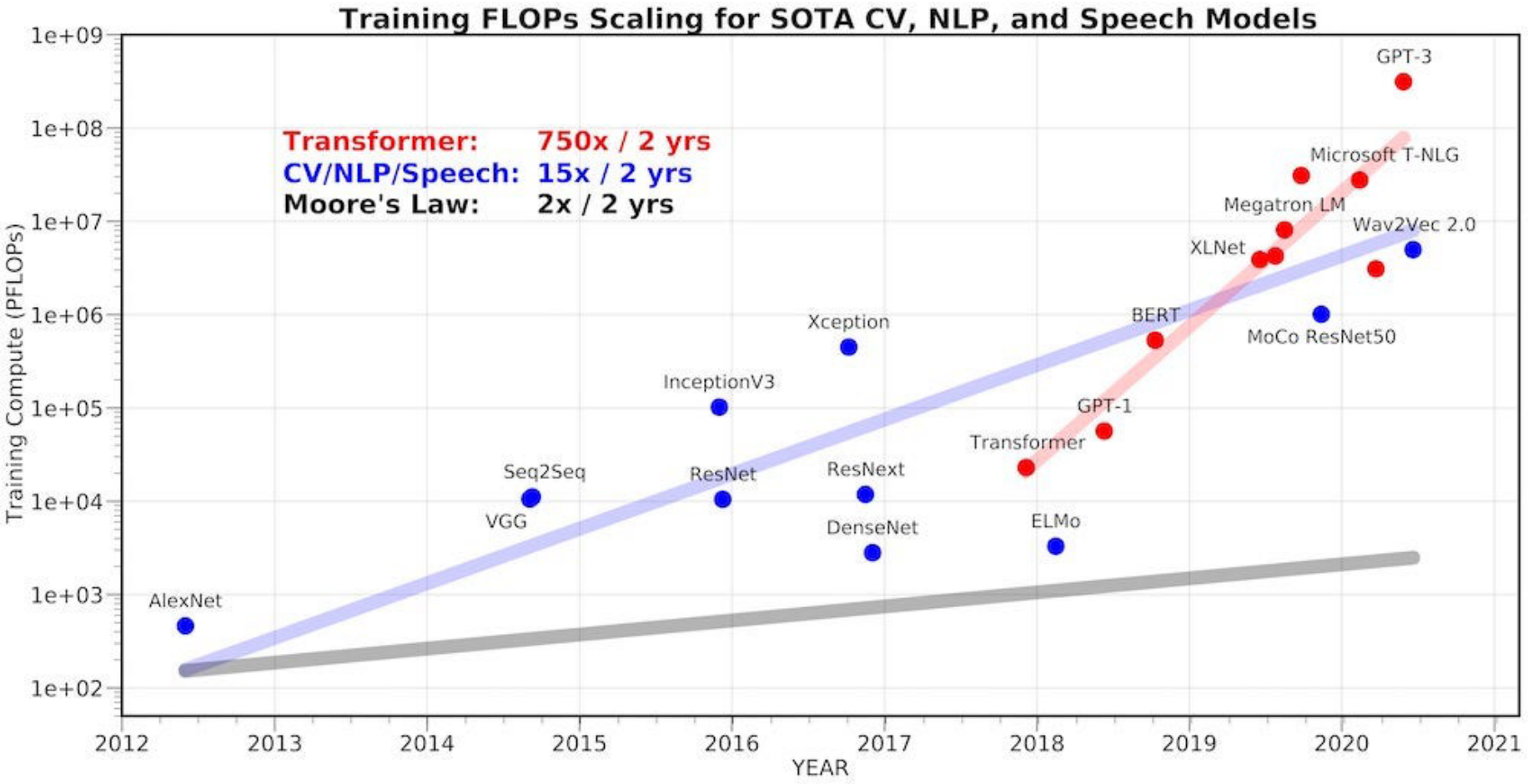


Moore’s Law

Moore's law is the observation that the number of transistors in an integrated circuit (IC) doubles about every two years.



Moore’s Law vs. AI



Performance Metrics

Response Time and Throughput

- **Response time (Latency)**
  - How long it takes to do a task
    - Important to individual users
- **Throughput**
  - Total work done per unit time
    - e.g., tasks/transactions/… per hour
    - Important to data center managers

Instruction Count and CPI

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles per Instruction}$$

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- Instruction Count for a program
  - Determined by program, ISA and compiler
- Average cycles per instruction
  - Determined by CPU hardware
  - If different instructions have different CPI
    - Average CPI affected by instruction mix

Performance Summary

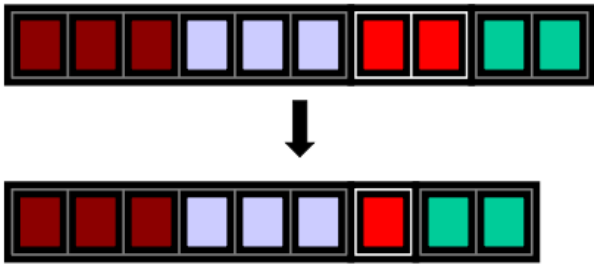
$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

	Instruction_ count	CPI	clock_cycle
Algorithm	X	X	
Programming language	X	X	
Compiler	X	X	
ISA	X	X	X
Core organization		X	X
Technology			X

Amdahl's Law

- Amdahl's law: performance enhancement possible with a given improvement is limited by the amount that the improved feature is used
- Pitfall: Improving an aspect of a computer and expecting a proportional improvement in overall performance

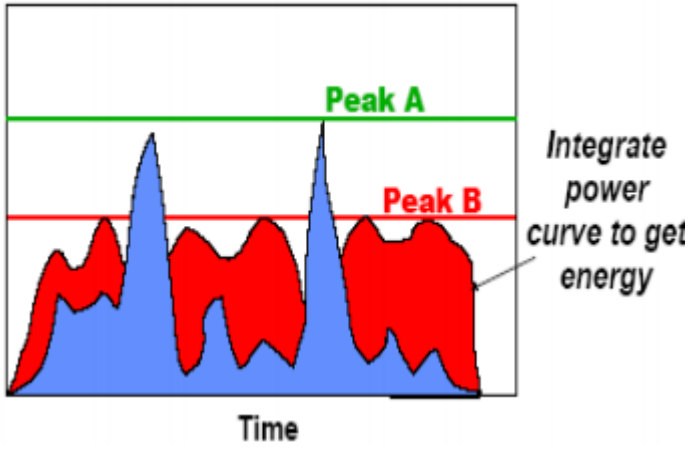
$$T_{improved} = \frac{T_{affected}}{improvement\ factor} + T_{unaffected}$$

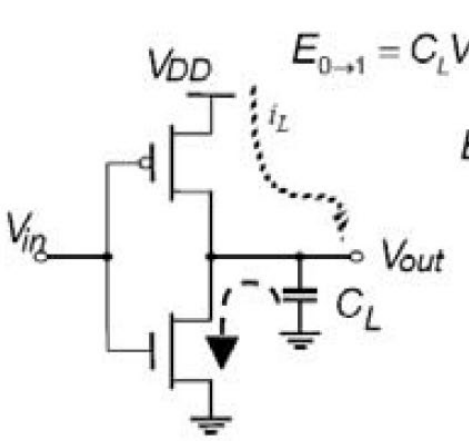


- Corollary: **make the common case fast**

Power and Energy

- Energy
  - measured in Joules
- Power
  - rate of energy consumption
    - [Watts = Joules/sec]
  - instantaneous power P = V * I





$$E_{0\to1} = C_L V_{DD}^2$$

$$E_R = \frac{1}{2} C_L V_{DD}^2$$

$$E_C = \frac{1}{2} C_L V_{DD}^2$$

- Energy consumed in N cycles, E_N:

$$E_N = C_L \bullet V_{DD}^2 \bullet n_{0\to1}$$

$n_{0\to1}$ – number of 0→1 transitions in N cycles

$$P_{avg} = \lim_{N\to\infty} \frac{E_N}{N} \cdot f = \left(\lim_{N\to\infty} \frac{n_{0\to1}}{N} \right) \cdot C_L \cdot V_{DD}^2 \cdot f$$

$$\alpha_{0\to1} = \lim_{N\to\infty} \frac{n_{0\to1}}{N} \cdot f$$

$$P_{avg} = \alpha_{0\to1} \cdot C_L \cdot V_{DD}^2 \cdot f$$

Dynamic Energy and Power

- Dynamic energy
  - Transistor switch from 0 -> 1 or 1 -> 0

$$\text{Energy}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2$$

- Dynamic power

$$\text{Power}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$$

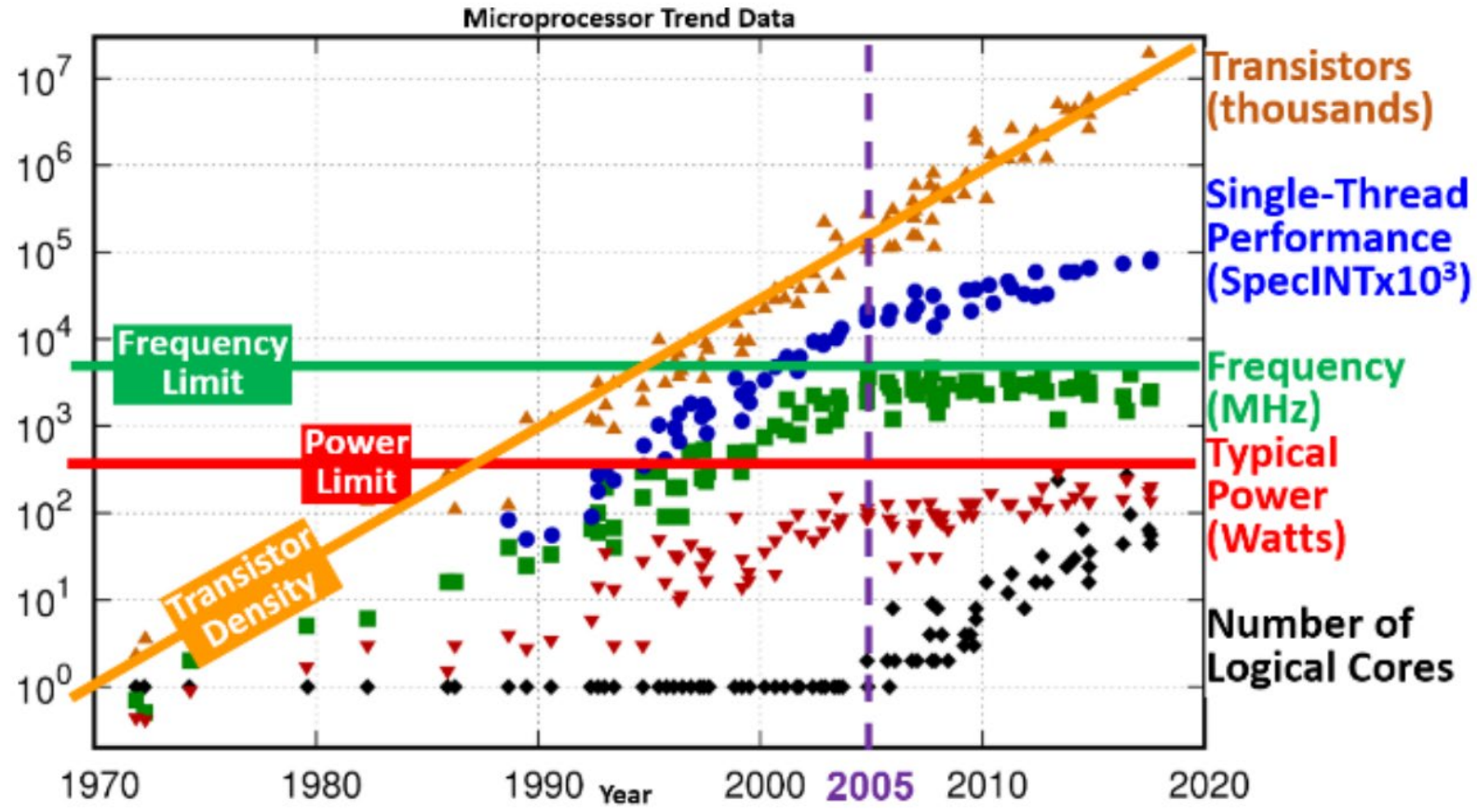
Reducing clock rate reduces power, not energy

- Static power consumption

$$\text{Power}_{\text{static}} \propto \text{Current}_{\text{static}} \times \text{Voltage}$$

  - Scales with number of transistors
  - To reduce: power gating

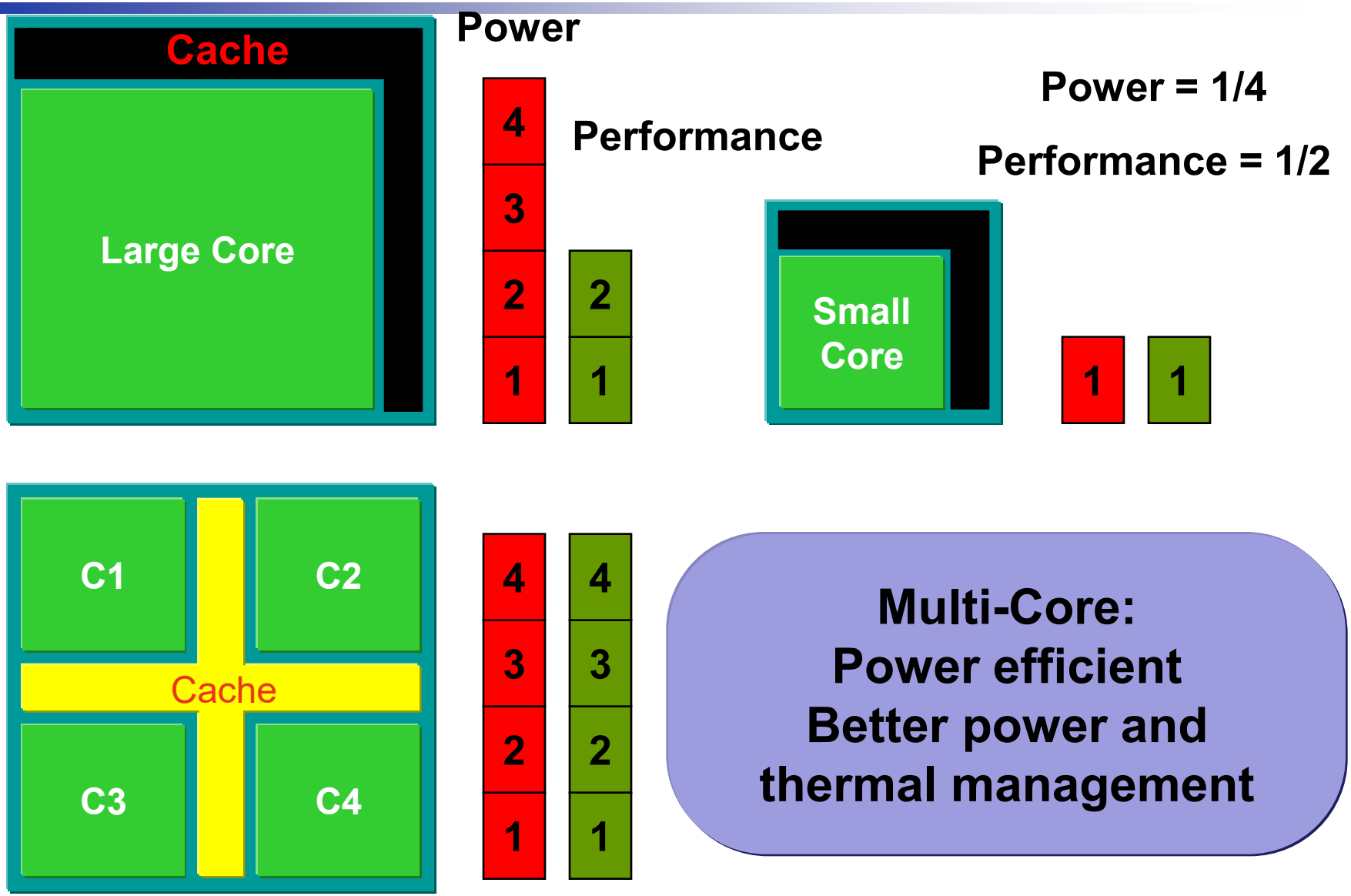
Power Trends and Power Wall



$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

$\times 30$ | $5\text{V} \rightarrow 1\text{V}$ | $\times 1000$

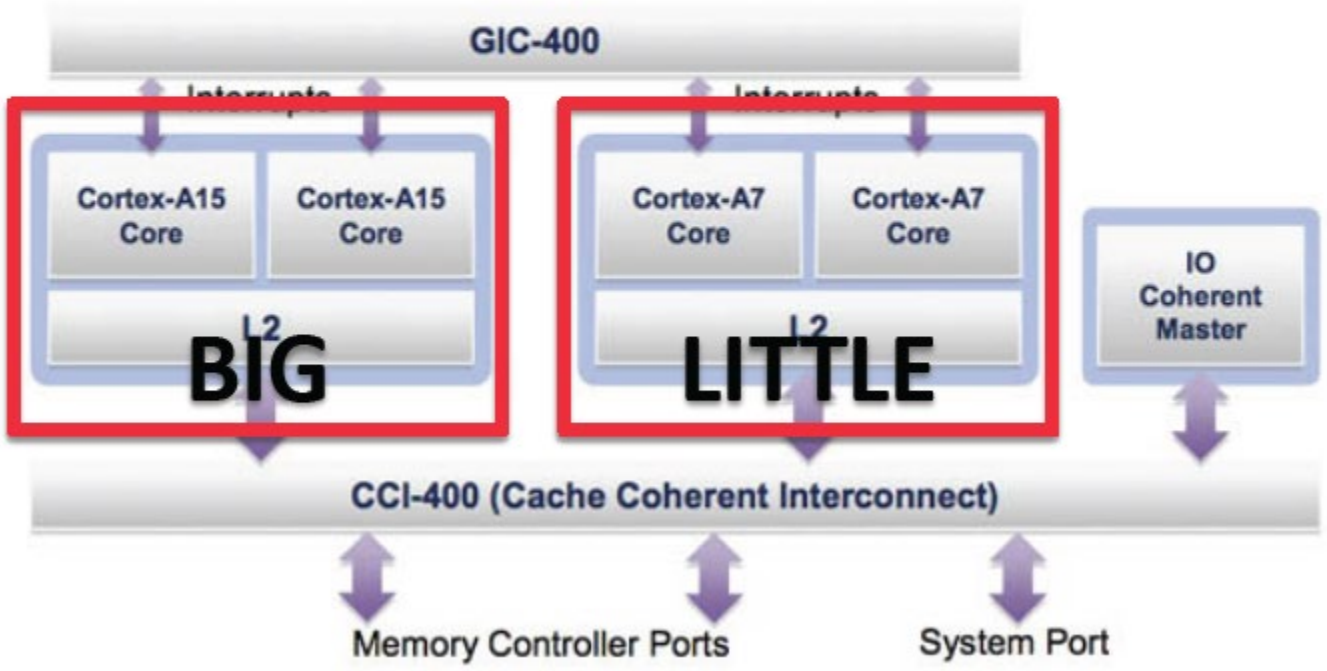
Why Multiple Cores on a Chip?



Multi-Core:
Power efficient
Better power and thermal management

Big Core vs Little Core

- What is big.LITTLE?
- **Big cores** for high performance
- **Little cores** for low power



Instructions: Language of the Computer

Instruction Set

- The repertoire of instructions of a computer
- Different computers have different instruction sets
  - But with many aspects in common
- Early computers had very simple instruction sets
  - Simplified implementation
- Many modern computers also have simple instruction sets

Design Principles

- *Design Principle 1:* Simplicity favours regularity
  - Regularity makes implementation simpler
  - Simplicity enables higher performance at lower cost
- *Design Principle 2:* Smaller is faster
  - Reason for small register file
  - c.f. main memory: millions of locations
- *Design Principle 3:* Make the common case fast
  - Small constants are common
  - Immediate operand avoids a load instruction
- *Design Principle 4:* Good design demands good compromises
  - Keep formats as similar as possible, 32-bit instructions
  - Different formats complicate decoding

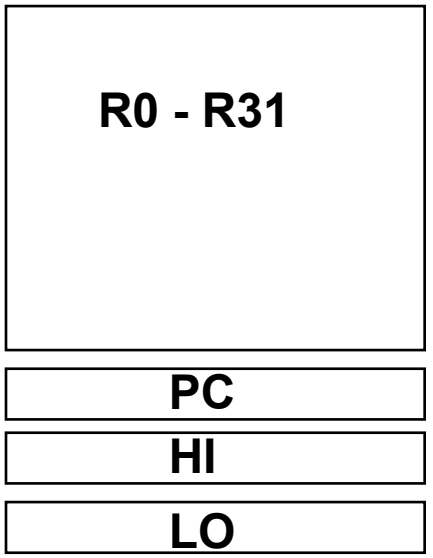
Memory Operands

- Main memory used for composite data
  - Arrays, structures, dynamic data, …
  - Only a small amount of data can fit in registers
- To apply arithmetic operations
  - Load values from memory into registers
  - Store result from register to memory
- Memory is byte addressed
  - Each address identifies an 8-bit byte
- Words are aligned in memory
  - Address must be a multiple of 4
- MIPS is Big Endian
  - Most-significant byte at least address of a word
  - *c.f.* Little Endian: least-significant byte at least address

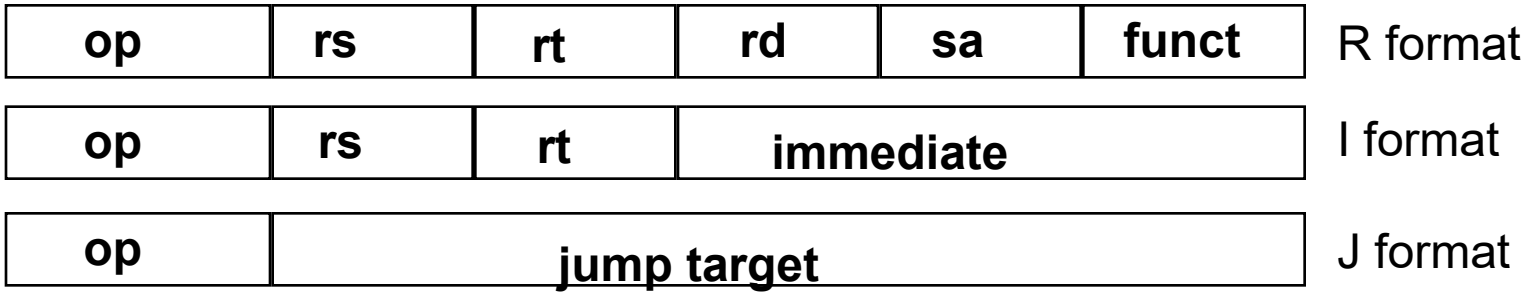
MIPS-32 ISA

- Instruction Categories
  1. Computational
  2. Load/Store
  3. Jump and Branch
  4. Floating Point
     --coprocessor
  5. Memory Management
  6. Special

Registers



Instruction Formats: all 32 bits wide

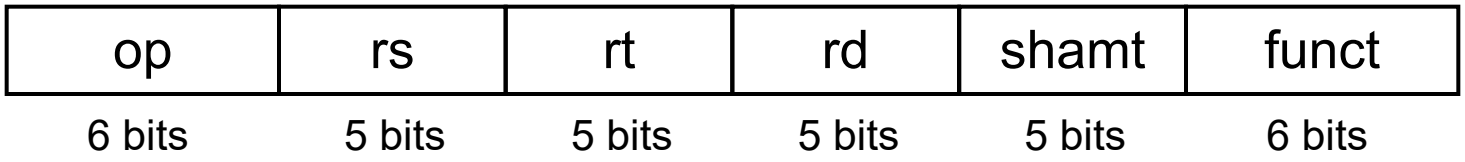


MIPS Register Conventions

Name	Register number	Usage	Preserved on call?
$zero	0	The constant value 0	n.a.
$v0–$v1	2–3	Values for results and expression evaluation	no
$a0–$a3	4–7	Arguments	no
$t0–$t7	8–15	Temporaries	no
$s0–$s7	16–23	Saved	yes
$t8–$t9	24–25	More temporaries	no
$gp	28	Global pointer	yes
$sp	29	Stack pointer	yes
$fp	30	Frame pointer	yes
$ra	31	Return address	yes

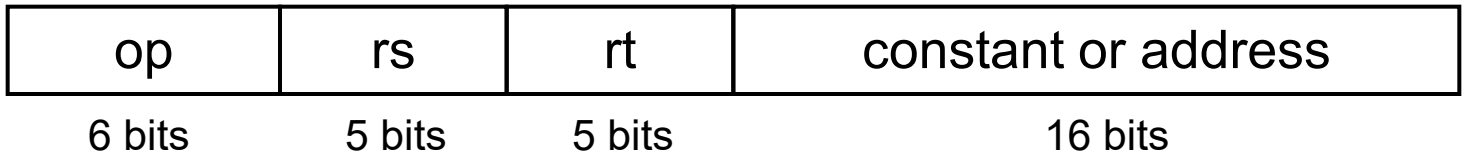
Register 1, called $at, is reserved for the assembler and registers 26−27, called $k0−$k1, are reserved for the operating system.

MIPS R-format Instructions



- Instruction fields
  - op: operation code (opcode)
  - rs: first source register number
  - rt: second source register number
  - rd: destination register number
  - shamt: shift amount (00000 for now)
  - funct: function code (extends opcode)

MIPS I-format Instructions



- Immediate arithmetic and load/store instructions
  - rt, rs: destination or source register number
  - Constant: -2^{15} to $+2^{15} - 1$
  - Address: offset added to base address in rs

The Constant Zero

- MIPS register 0 ($zero) is the constant 0
  - Cannot be overwritten
- Useful for common operations
  - E.g., move between registers

```
add $t2, $s1, $zero
addi $t2, $zero,100
```

Conditional Operations

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- `beq rs, rt, L1`
  - if (rs == rt) branch to instruction labeled L1;
- `bne rs, rt, L1`
  - if (rs != rt) branch to instruction labeled L1;
- `j L1`
  - unconditional jump to instruction labeled L1

More Conditional Operations

- Set result to 1 if a condition is true
  - Otherwise, set to 0
- `slt rd, rs, rt`
  - if (rs < rt) rd = 1; else rd = 0;
- `slti rt, rs, constant`
  - if (rs < constant) rt = 1; else rt = 0;
- Use in combination with beq, bne

```
slt $t0, $s1, $s2  # if ($s1 < $s2)
bne $t0, $zero, L  #   branch to L
```

Sign Extension

- Examples: 8-bit to 16-bit
  - +2: 0000 0010 => 0000 0000 0000 0010
  - –2: 1111 1110 => 1111 1111 1111 1110
- In MIPS instruction set
  - `addi`: extend immediate value
  - `lb`, `lh`: extend loaded byte/halfword
  - `beq`, `bne`: extend the displacement

```
lb rt, offset(rs)     lh rt, offset(rs)
```

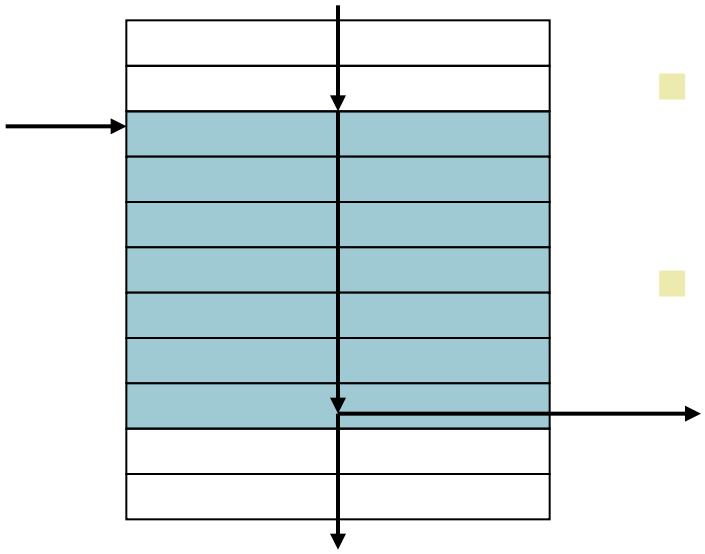
  - **Sign extend to 32 bits in rt**

```
sb rt, offset(rs)     sh rt, offset(rs)
```

  - **Store just rightmost byte/halfword**

Basic Blocks

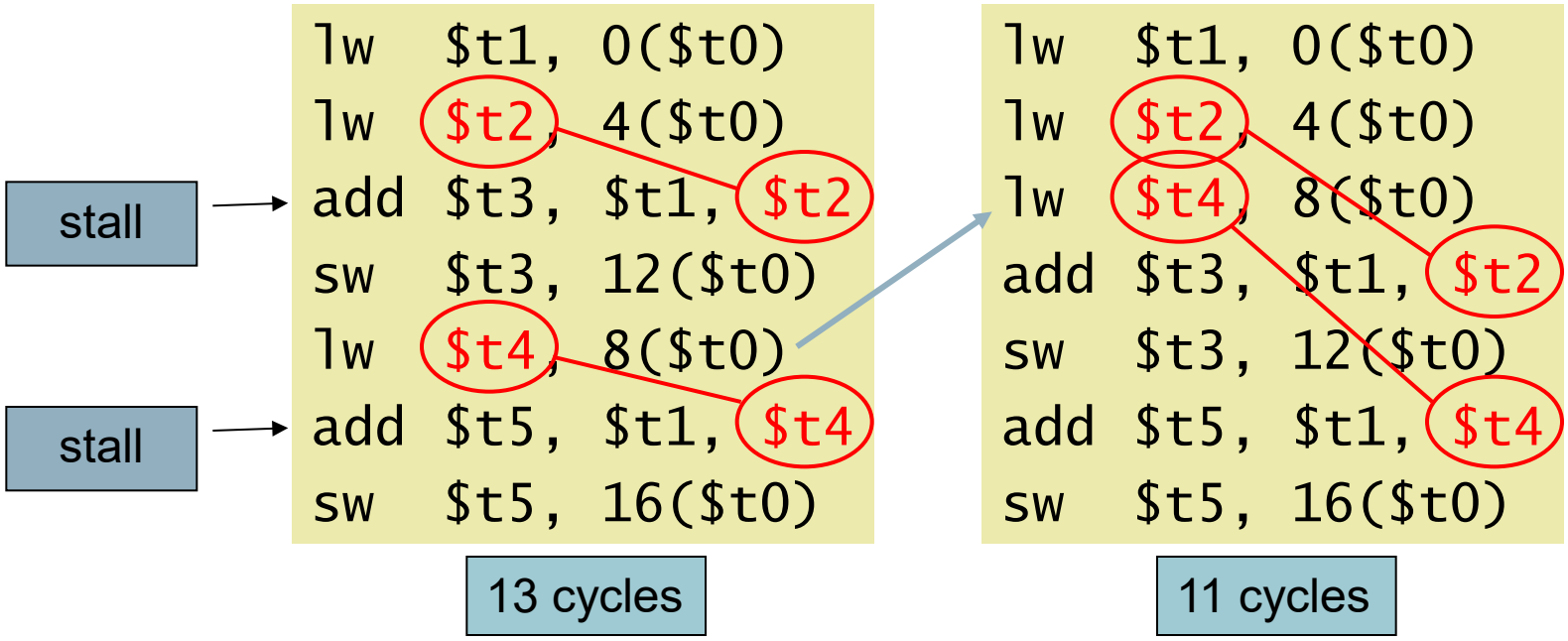
- A basic block is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets (except at beginning)



- A compiler identifies basic blocks for optimization
- An advanced processor can accelerate execution of basic blocks

Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for `A = B + E; C = B + F;`



Procedure Calling

- Procedures enable structured programs
  - Easier to understand and reuse code
- Steps required
  1. Place parameters in registers
  2. Transfer control to procedure
  3. Acquire storage for procedure
  4. Perform procedure's operations
  5. Place result in register for caller
  6. Return to place of call

Procedure Call Instructions

- Procedure call: jump and link

  `jal ProcedureLabel`

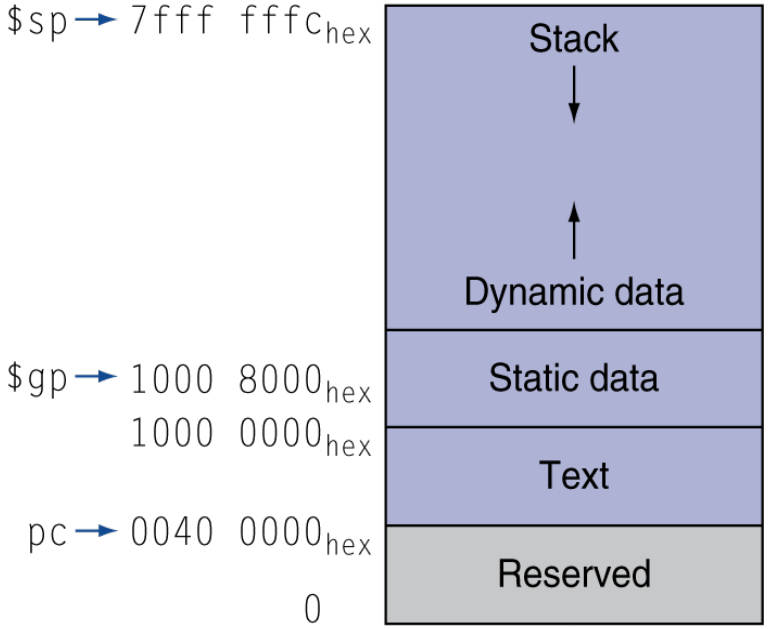
  - Address of following instruction put in $ra
  - Jumps to target address
- Procedure return: jump register

  `jr $ra`

  - Copies $ra to program counter
  - Can also be used for computed jumps
    - e.g., for case/switch statements

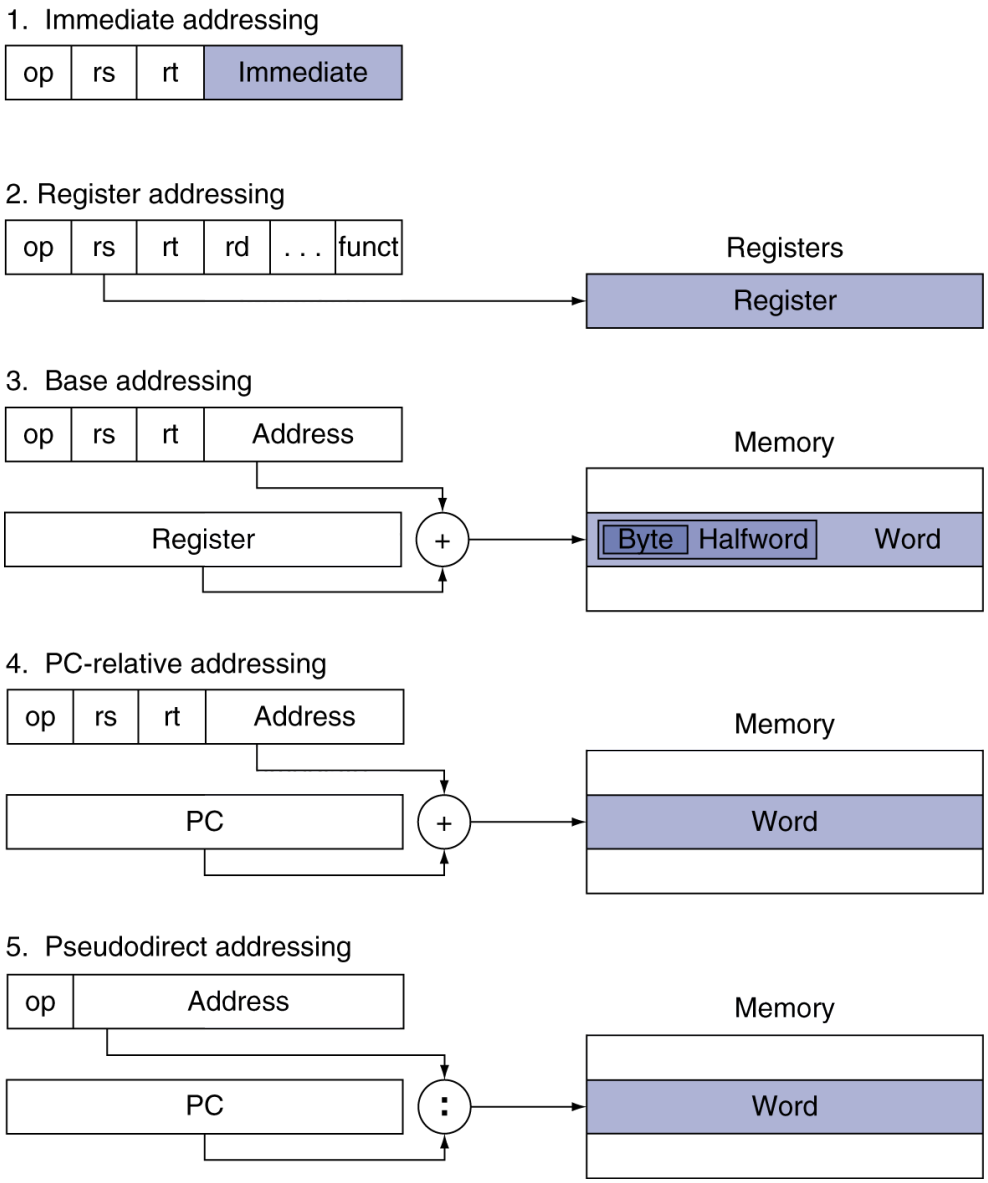
Memory Layout

- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - $gp initialized to address allowing ±offsets into this segment
- Dynamic data: heap
  - E.g., malloc() in C, new in Java, linked lists
  - Must use free() in C
    - Memory leak issues!
- Stack: automatic storage



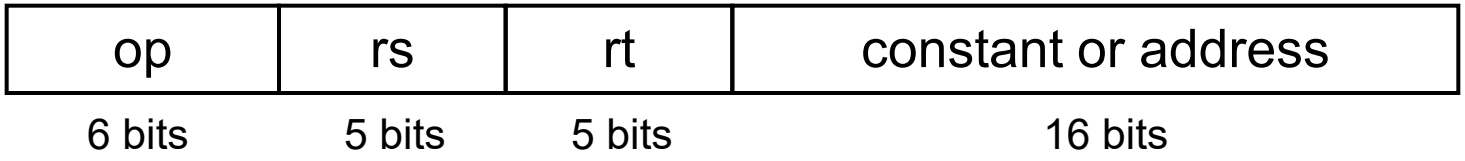
Addressing

Addressing Mode Summary



Branch Addressing

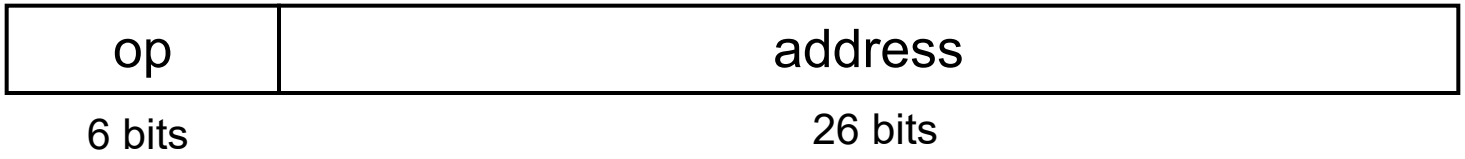
- Branch instructions specify
  - Opcode, two registers, target address
- If addresses had to fit in 16-bits, then no program could be bigger than 2^{16}



- PC-relative addressing
  - Target address = PC + 4 + offset × 4
  - Almost all loops and *if* statements < 2^{16} words

Jump Addressing

- Jump (`j` and `jal`) targets could be anywhere in text segment
  - Encode full address in instruction



- (Pseudo)Direct jump addressing
  - Target address = $PC_{31\ldots28}$: (address × 4)

Target Addressing Example

- Loop code from earlier example
  - Assume Loop at location 80000

```
Loop: sll  $t1, $s3, 2      80000
      add  $t1, $t1, $s6    80004
      lw   $t0, 0($t1)      80008
      bne  $t0, $s5, Exit   80012
      addi $s3, $s3, 1      80016
      j    Loop             80020
Exit: …                     80024
```

0	0	19	9	4	0
0	9	22	9	0	32
35	9	8	0		
5	8	21	2		
8	19	19	1		
2	20000				

Branching Far Away

- If branch target is too far to encode with 16-bit offset, assembler rewrites the code
- Example: how can we increase branching distance for the following instruction?

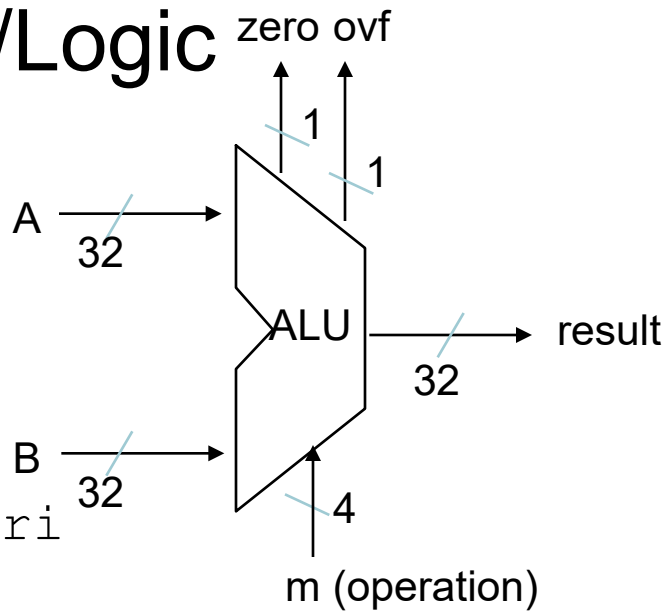
```
        beq $s0,$s1, L1
              //(如果相等，则跳转，但是L1超出了16位地址范围)
                 ↓
        bne $s0,$s1, L2  //可以转换为无条件跳转
        j L1
L2:     …
```

Arithmetic for Computers

MIPS Arithmetic Logic Unit (ALU)

- Must support the Arithmetic/Logic operations of the ISA

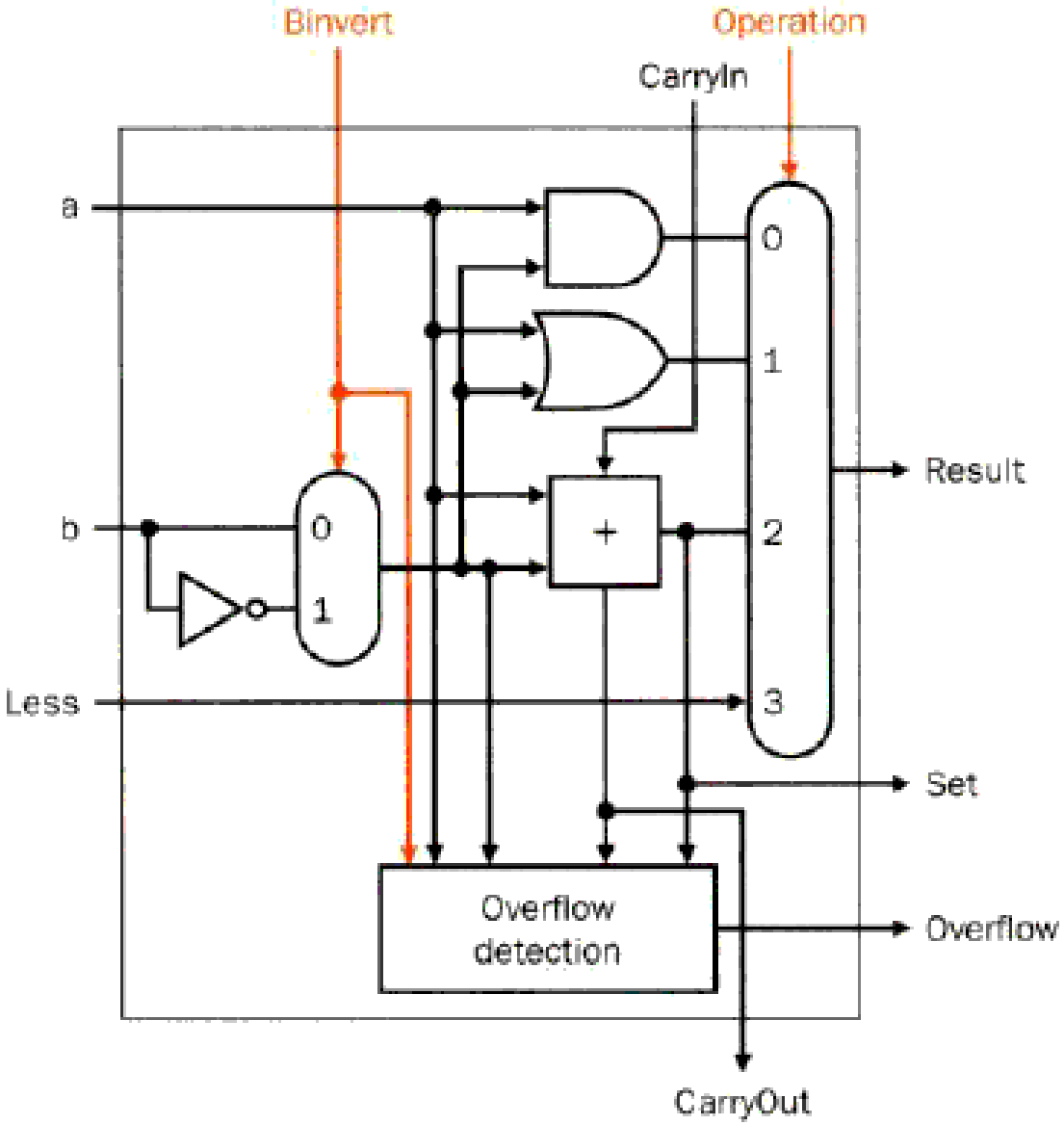
```
add, addi, addiu, addu
sub, subu
mult, multu, div, divu
sqrt
and, andi, nor, or, ori, xor, xori
beq, bne, slt, slti, sltiu, sltu
```



- With special handling for
  - sign extend – `addi, addiu, slti, sltiu`
  - zero extend – `andi, ori, xori`
  - overflow detection – `add, addi, sub`

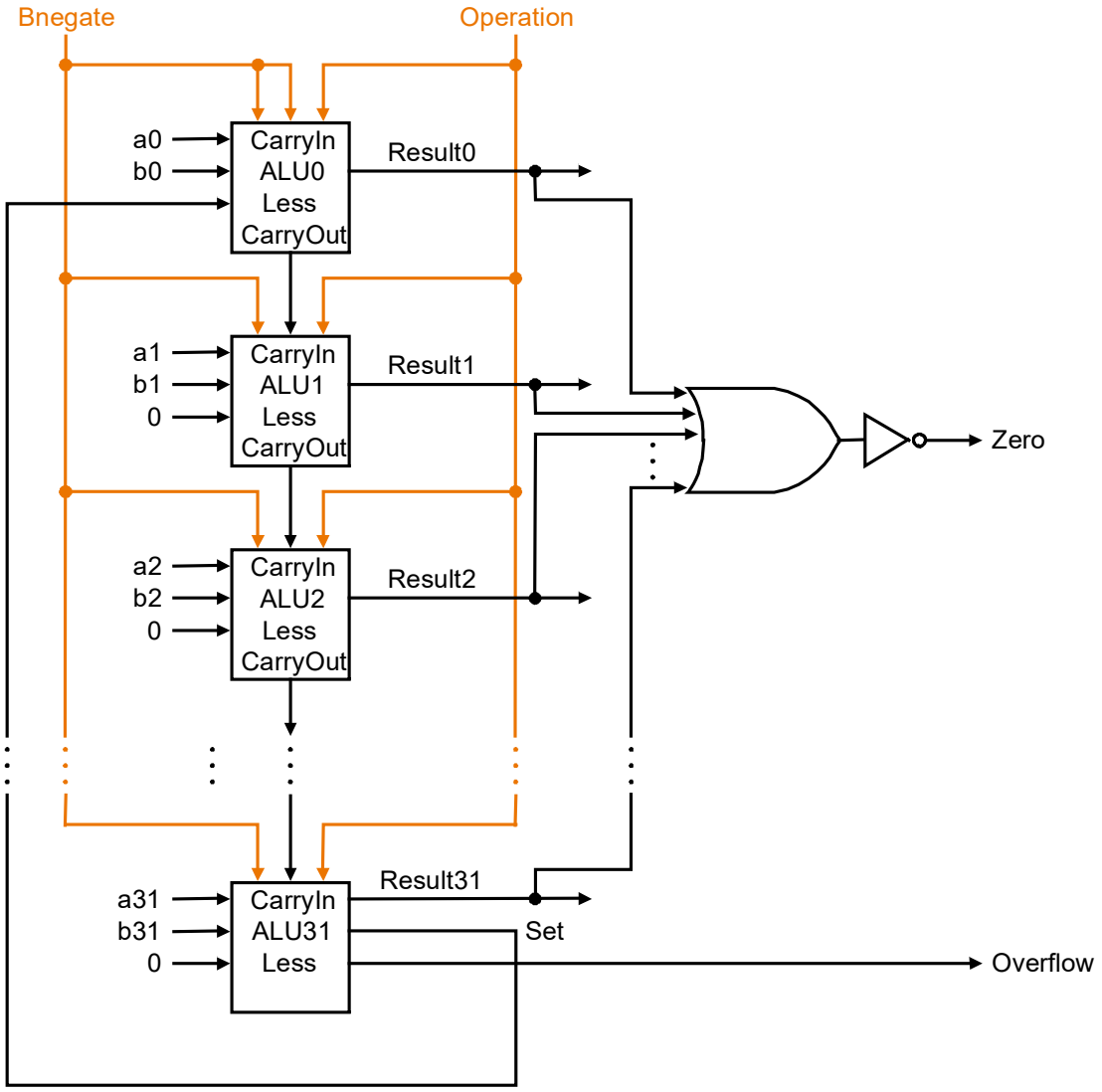
Review Appendix B (from CD or lecture page) for more details on ALU design

1-bit ALU


Binvert
Operation
CarryIn
a
b
Less
0
1
+
0
1
2
3
Result
Set
Overflow
detection
Overflow
CarryOut

Complete ALU

- Input
  - A
  - B
- Control lines
  - Binvert
  - Operation
  - Carryin
- Output
  - Result
  - Overflow
  - Zero detector



Multiplication

- Start with long-multiplication approach

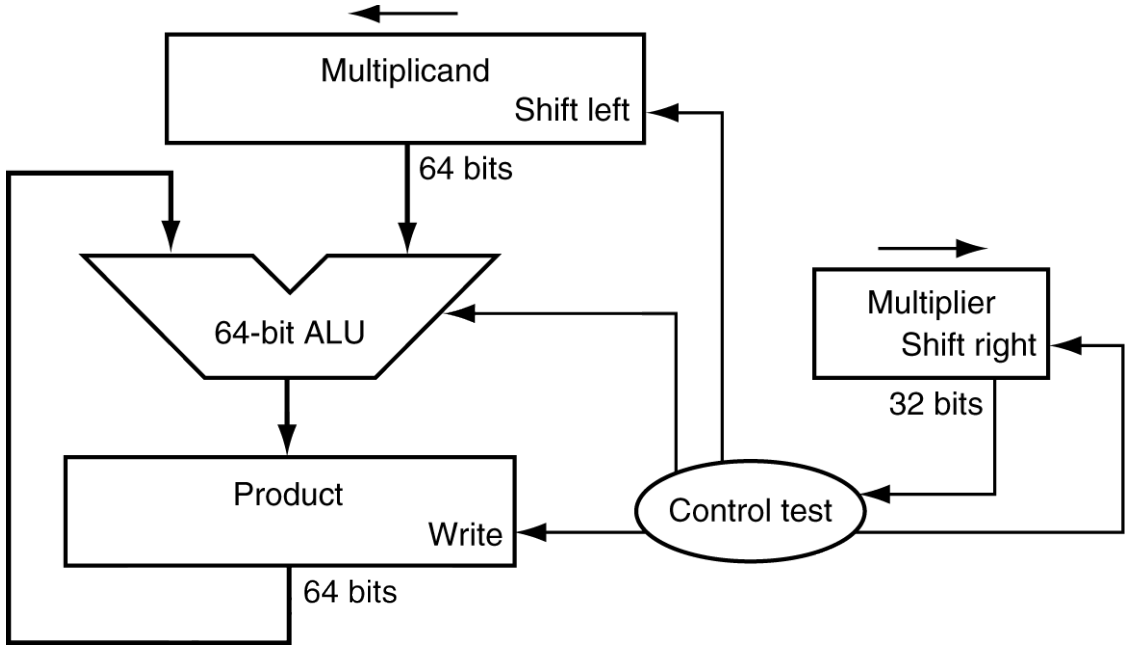
multiplicand

multiplier

product

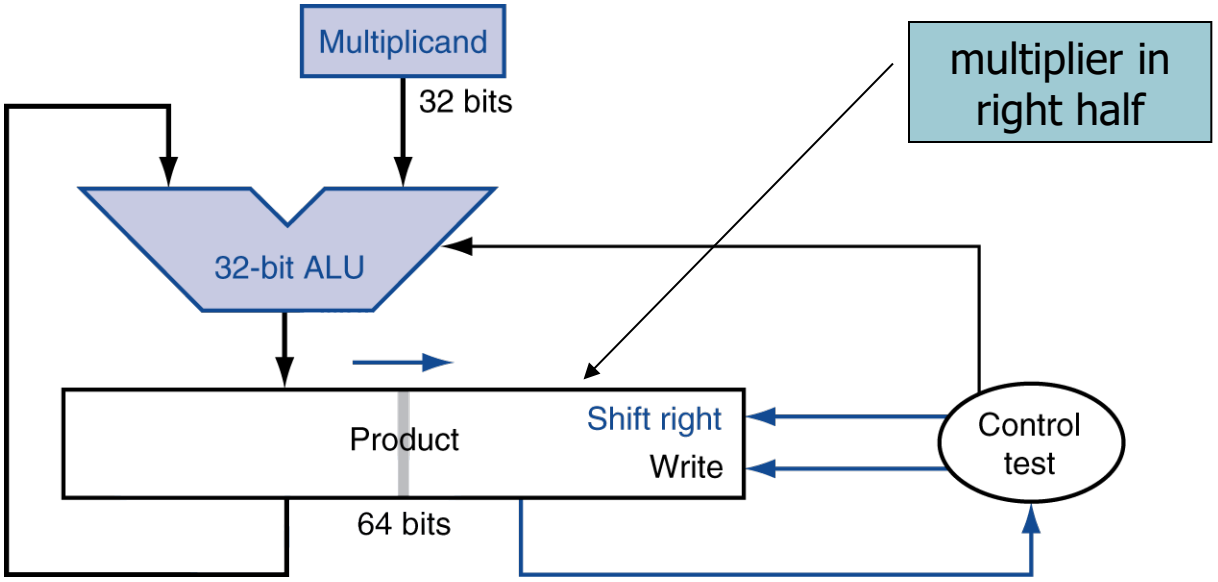
```
   1000
 × 1001
 -------
   1000
  0000
 0000
1000
 -------
01001000
```

Length of product is the sum of operand lengths



Optimized Multiplier

- Perform steps in parallel: add/shift



- One cycle per partial-product addition
  - That's ok, if frequency of multiplications is low
  - ~32 cycles for multiplying 2 32-bit values

MIPS Multiplication

- Two 32-bit registers for product
  - HI: most-significant 32 bits
  - LO: least-significant 32-bits
- Instructions
  - `mult rs, rt  /  multu rs, rt`
    - 64-bit product in HI/LO
  - `mfhi rd  /  mflo rd`
    - Move from HI/LO to rd
    - Can test HI value to see if product overflows 32 bits
  - `mul rd, rs, rt`
    - Least-significant 32 bits of product –> rd

恢复余数法

运算规则：

1）被除数减除数，够减时商**1**；不够减时商**0**。

2）商 **0** 时不能作减法，为了下次能正确运算，

必须把已减掉的除数加回去，恢复余数。

3) 最后余数与除数的符号位保持一致。

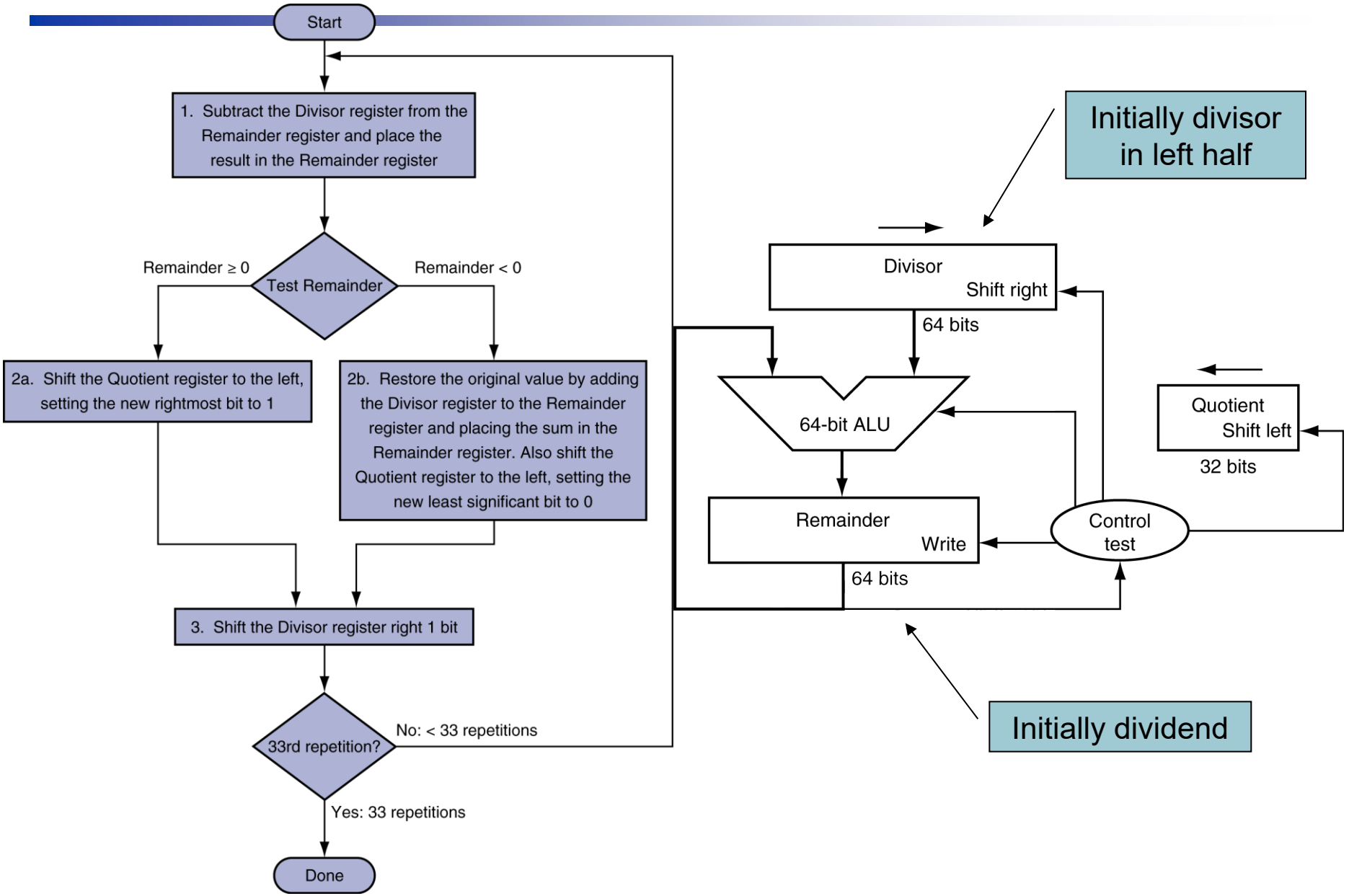
这就是“恢复余数法”的基本思想。

```
        1001
1000 ) 1001010
      -1000
         10
         101
         1010
        -1000
          10
```

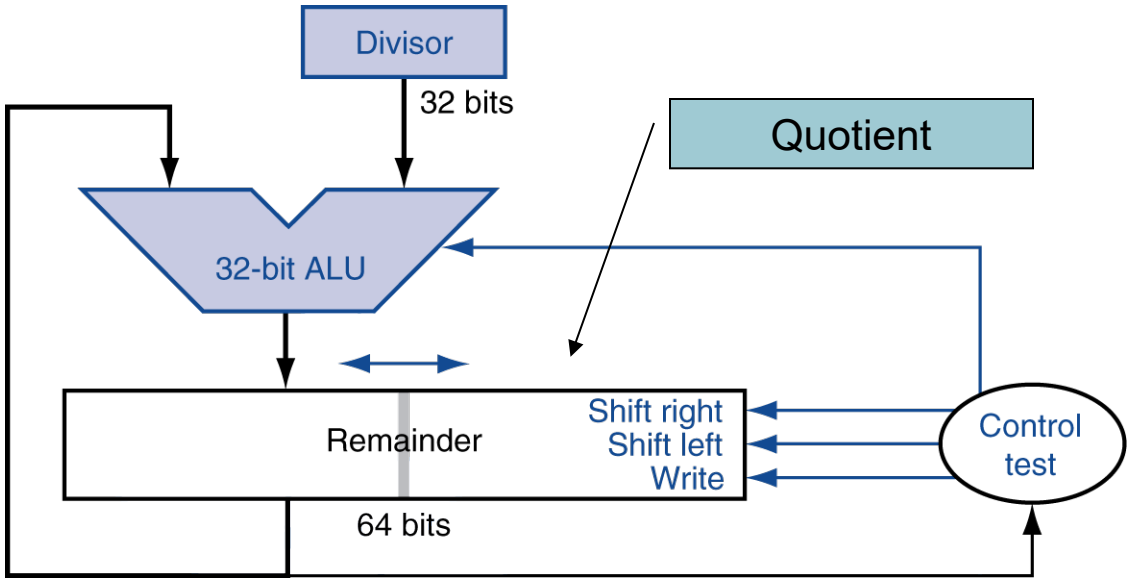
计算 x/y的过程: x= $[0111]_2$ y=$[0010]_2$

迭代次数	步骤	商	除数	余数
0	初始化	0000	**0010 0000**	0000 0111
1 比较余数与 y * 2^4	余数=余数-除数 试商 余数<0**商左移**1位，**最低位置**0 除数右移	0000 0000 0000	0010 0000 0010 0000 0001 0000	1110 0111 **0000 0111** 0000 0111
2	余数=余数-除数 试商 余数<0商左移1位，**最低位置**0 除数右移	0000 0000 0000	**0001 0000** 0001 0000 0000 1000	1111 0111 **0000 0111** 0000 0111
3	余数=余数-除数 试商 余数<0商左移1位，**最低位置**0 除数右移	0000 0000 0000	**0000 1000** 0000 1000 0000 0100	1111 1111 **0000 0111** 0000 0111
4	余数=余数-除数 试商 余数>0商左移1位，最低置1 除数右移	0000 0001 0001	**0000 0100** 0000 0100 0000 0010	0000 0011 0000 0011 0000 0011
5 比较余数与 y * 2^0	余数=余数-除数 试商 余数>0商左移1位，**最低位置**1 除数右移	0000 0010 0011	**0000 0010** 0000 0010 0000 0001	0000 0001 0000 0001 0000 0001

Integer Division Hardware



Optimized Integer Divider



- One cycle per partial-remainder subtraction
- Looks a lot like a multiplier!
  - Same hardware can be used for both

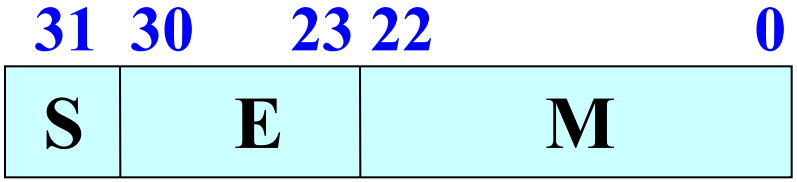
MIPS Division

- Use HI/LO registers for result
  - HI: 32-bit remainder
  - LO: 32-bit quotient
- Instructions
  - `div rs, rt` / `divu rs, rt`
  - Use `mfhi`, `mflo` to access result
  - No overflow or divide-by-0 checking
    - Software must perform checks if required

Floating Point

规格化浮点数的真值

32位浮点数格式：



移码定义：$[x]_{移} = x0\ x1\ x2 \cdots xn$

$= 2^n + x \qquad -2^n \le x < 2^n$

一个规格化的32位浮点数 x 的真值为：

$$x = (-1)^s \times (1.M) \times 2^{E-127} \qquad e = E - 127$$

一个规格化的64位浮点数 x 的真值为：

$$x = (-1)^s \times (1.M) \times 2^{E-1023}$$

这里e是真值， E 是机器数

完成浮点加减运算的操作过程大体分为六步：

(1) 0 操作数的检查；

(2) 比较阶码大小并完成对阶；

(3) 尾数进行加或减运算；

(4) 结果规格化；

(5) 舍入处理；

(6) 溢出处理。

$$x \pm y = (M_x 2^{Ex-Ey} \pm M_y) 2^{Ey} \qquad E_x <= E_y$$

结果规格化

求和之后得到的数可能不是规格化了的数，为了增加有效数字的位数，提高运算精度，必须将求和的结果规格化.

①规格化的定义：$\frac{1}{2} \le |S| < 1$ （二进制）

采用原码：

正数：S=0.1 ×××…×

负数：S=1.1 ×××…×

采用双符号位的补码：

对正数：S=00.1×××…×

对负数：S=11.0×××…×

②向左规格化

若不是规格化的数，需要尾数向左移位，以实现规格化的过程，我们称其为向左规格化。

前例中，00 11，11.1001不是规格化数，因而需要左规，即左移一位，阶码减1，得：

$$[x+y]_{补}=00\ 10,\ 11.0010$$

③向右规格化

浮点加减运算时，尾数求和的结果也可能得到：

01.×××…×　或　10.×××…×，

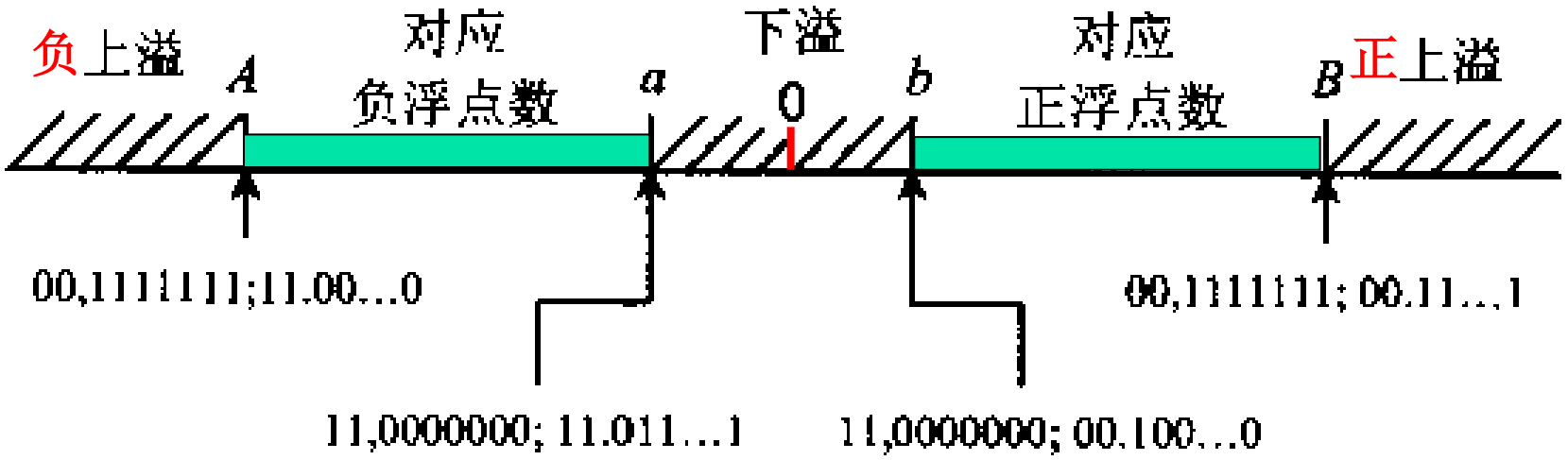
即两符号位不等，即结果的绝对值大于1。向左破坏了规格化。

此时，将尾数运算的结果右移一位，阶码加1，称为向右规格化。

溢出处理

与定点加减法一样，浮点加减运算最后一步也需判溢出。在浮点规格化中已指出，当尾数之和(差)出现01．××…×或10．××…×时，并不表示溢出，只有将此数右规后，再根据阶码来判断浮点运算结果是否溢出。

若机器数为补码，尾数为规格化形式，并假设阶符取2位，阶码取7位、数符取2位，尾数取n位，则它们能表示的补码在数轴上的表示范围如图所示。



Some specific numbers

0: 0000 0000 … 0000

–1: 1111 1111 … 1111

Most-negative: 1000 0000 … 0000

Most-positive: 0111 1111 … 1111

图中a,b之间的阴影部分，对应阶码小于128的情况，叫做浮点数的下溢。下溢时．浮点数值趋于零，故机器不做溢出处理，仅把它作为机器零。

图中的A、B两侧阴影部分，对应阶码大于127的情况，叫做浮点数的上溢。此刻，浮点数真正溢出，机器需停止运算，作溢出中断处理。一般说浮点溢出，均是指上溢。

可见，浮点机的溢出与否可由阶码的符号决定：

阶码$[j]_{补}$=01,×××××为上溢，机器停止运算，做中断处理；

阶码$[j]_{补}$=10,×××××为下溢，按机器零处理。

浮点乘、除法运算

1. 浮点乘法、除法运算规则

设有两个浮点数 x 和 y：

$$x = 2^{Ex} \cdot M_x$$
$$y = 2^{Ey} \cdot M_y$$

浮点乘法运算的规则是：

$$x \times y = 2^{(Ex+Ey)} \cdot (M_x \times M_y)$$

即：乘积的尾数是相乘两数的尾数之积；
乘积的阶码是相乘两数的阶码之和。

浮点除法运算的规则是：

$$x \div y = 2^{(Ex-Ey)} \cdot (M_x \div M_y)$$

即：商的尾数是相除两数的尾数之商；
商的阶码是相除两数的阶码之差。

2. 浮点乘、除法运算步骤

浮点数的乘除运算大体分为四步：

(1) 0 操作数检查；

(2) 阶码加/减操作；

(3) 尾数乘/除操作；

(4) 结果规格化及舍入处理。

FP Instructions in MIPS

- FP hardware is coprocessor 1
  - Adjunct processor that extends the ISA
- Separate FP registers
  - 32 single-precision: $f0, $f1, … $f31
  - Paired for double-precision: $f0/$f1, $f2/$f3, …
    - Release 2 of MIPs ISA supports 32 × 64-bit FP reg's
- FP instructions operate only on FP registers
  - Programs generally don't do integer ops on FP data, or vice versa
  - More registers with minimal code-size impact
- FP load and store instructions
  - `lwc1`, `ldc1`, `swc1`, `sdc1`
    - e.g., `ldc1 $f8, 32($sp)`

FP Instructions in MIPS

- Single-precision arithmetic
  - `add.s`, `sub.s`, `mul.s`, `div.s`
    - e.g., `add.s $f0, $f1, $f6`
- Double-precision arithmetic
  - `add.d`, `sub.d`, `mul.d`, `div.d`
    - e.g., `mul.d $f4, $f4, $f6`
- Single- and double-precision comparison
  - `c.`*xx*`.s`, `c.`*xx*`.d` (*xx* is `eq`, `lt`, `le`, ...)
  - Sets or clears FP condition-code bit
    - e.g. `c.lt.s $f3, $f4`
- Branch on FP condition code true or false
  - `bc1t`, `bc1f`
    - e.g., `bc1t TargetLabel`

Accurate Arithmetic

- Infinite variety of real numbers between, say, 0 and 1
  - Only 2^{53} can be represented by double precision FP
- IEEE Std 754 specifies additional rounding control
  - Extra bits of precision (guard, round, sticky)
  - guard and round bits are 2 extra bits kept on the right during intermediate additions
  - sticky bit used in rounding in addition to guard and round bits; is set whenever a 1 bit shifts right of the round bit

F = 1 . xxxxxxxxxxxxxxxxxxxxxxx G R S

- Not all FP units implement all options
  - Most programming languages and FP libraries just use defaults
- Trade-off between hardware complexity, performance, and market requirements

Parallelism and Associativity

- Parallel programs may interleave operations in unexpected orders
  - Integer addition is associative
  - Assumptions of associativity for FP numbers may fail!

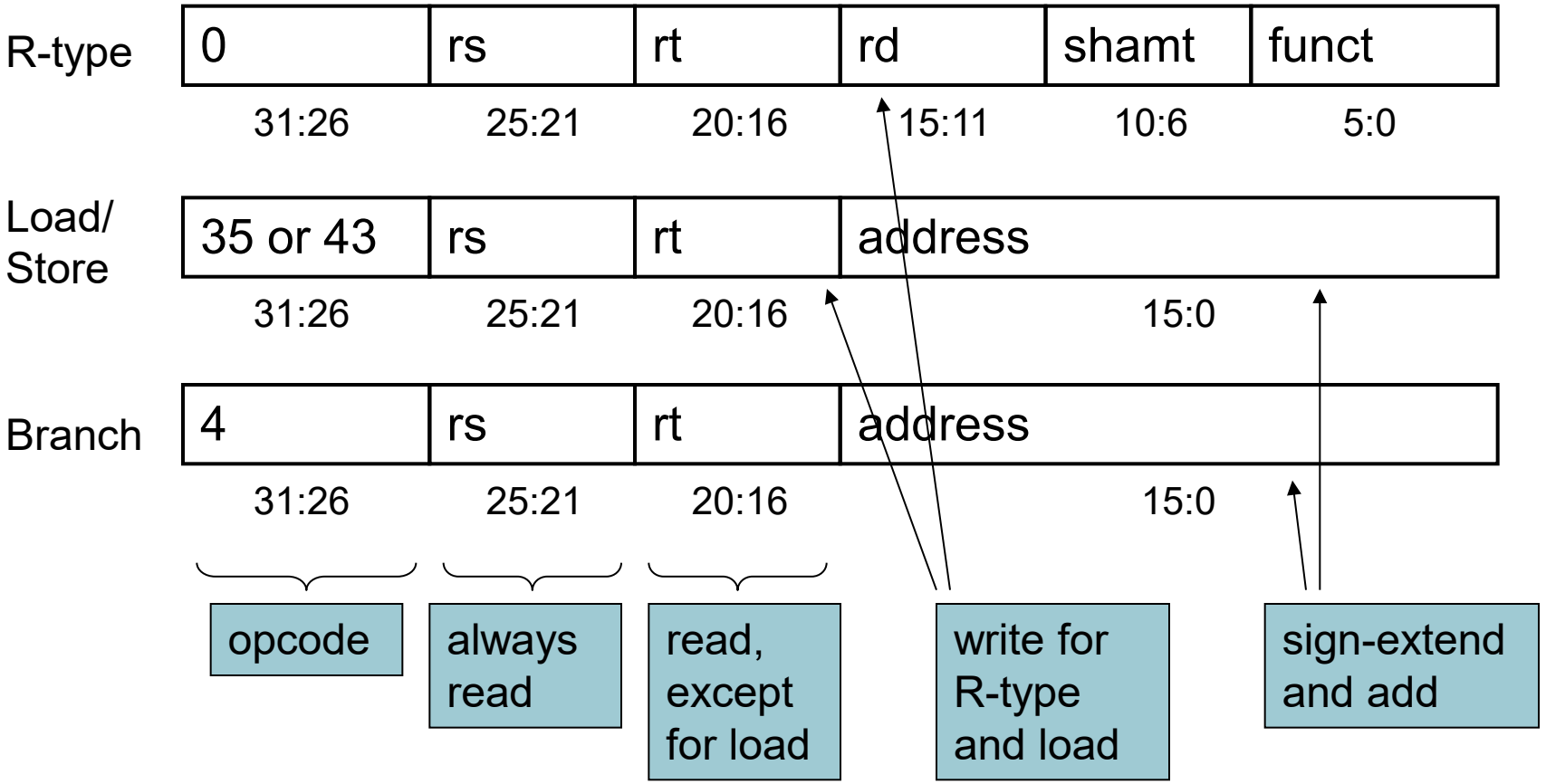
		(x+y)+z	x+(y+z)
x	-1.50E+38		-1.50E+38
y	1.50E+38	0.00E+00	
z	1.0	1.0	1.50E+38
		1.00E+00	0.00E+00

- Floating point numbers are approximations of real numbers – not associative!
- Need to validate parallel programs under varying degrees of parallelism

The Processor

The Main Control Unit

- Control signals derived from instruction

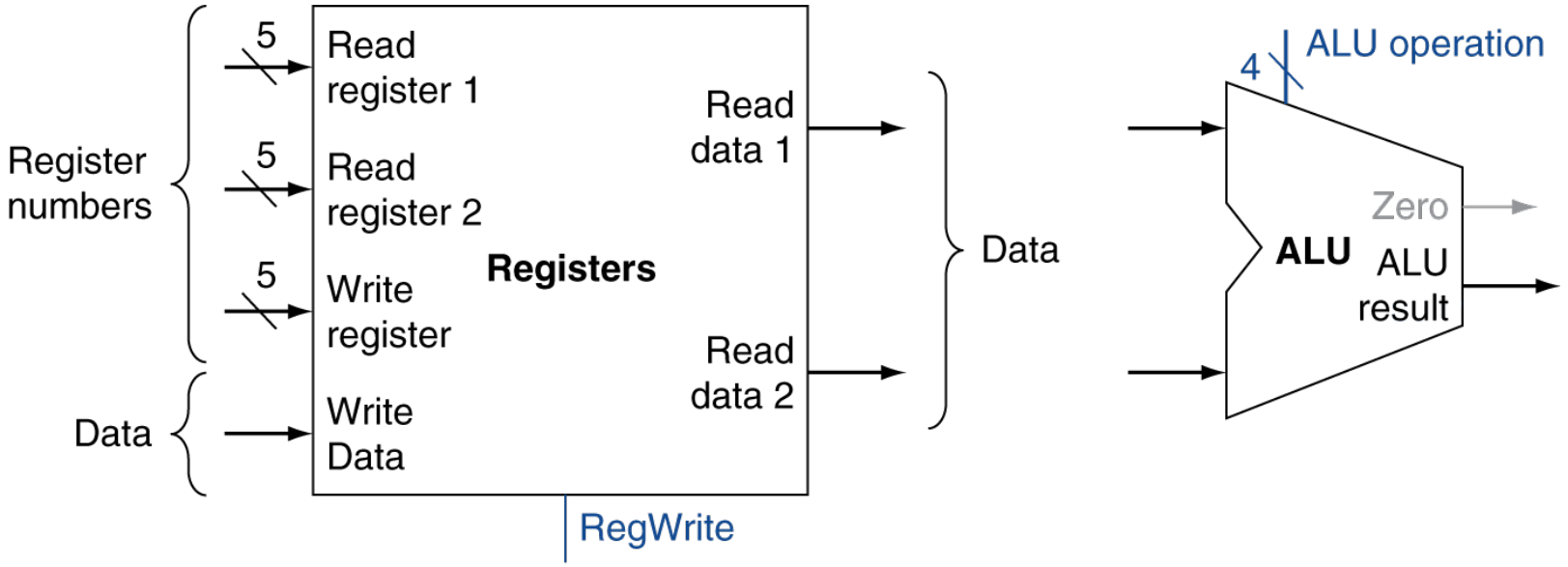


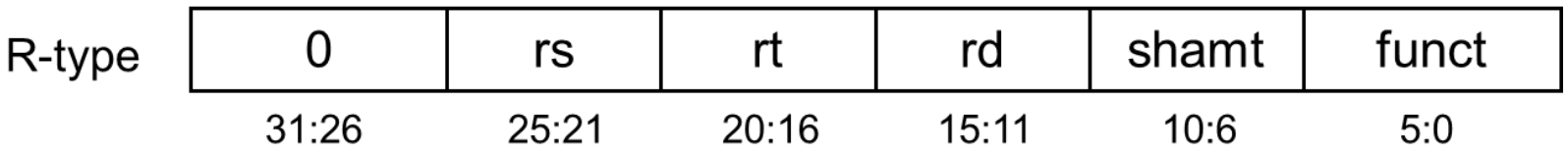
Instruction Execution: Main Steps

- PC → instruction memory, fetch instruction
- Register numbers → register file, read registers
- Depending on instruction class
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch target address
  - Write result to register file (optional)
  - Access data memory for load/store (optional)
  - PC ← target address or PC + 4

R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

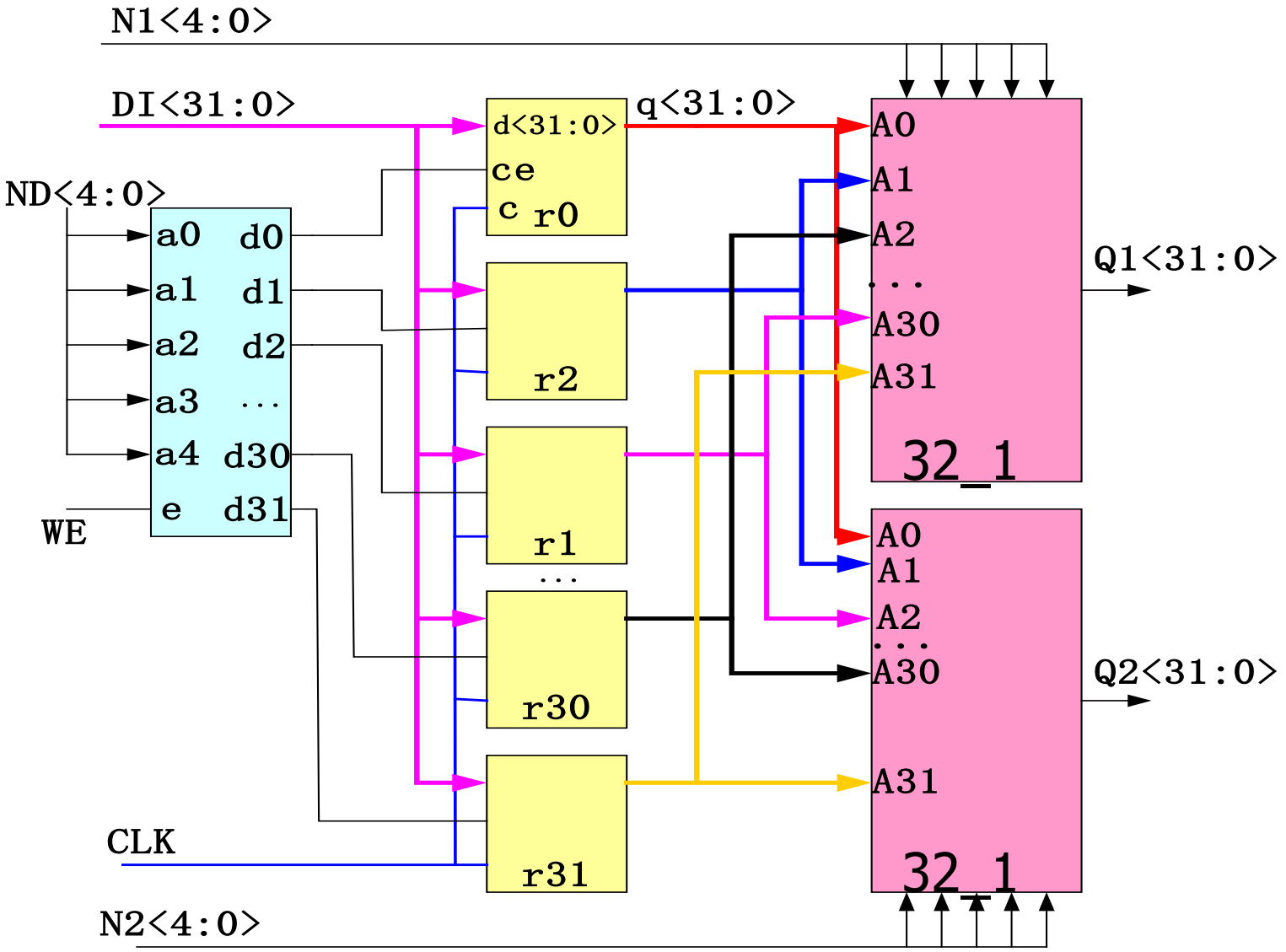




Register File

```
add $t0, $s1, $s2
```

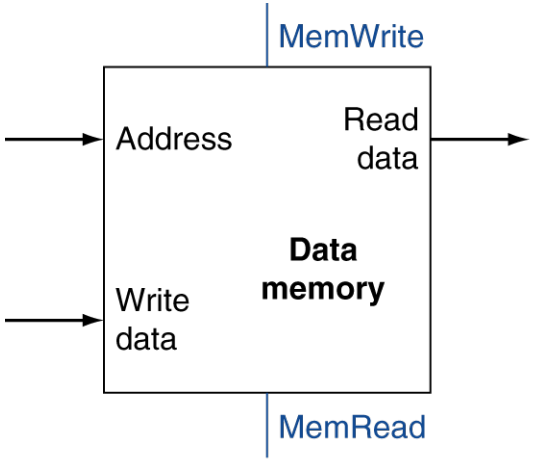
special	$s1	$s2	$t0	0	add



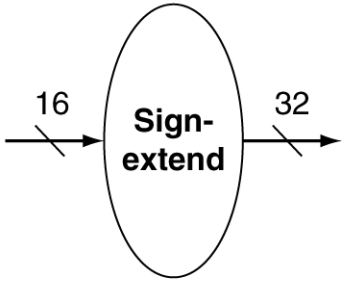
32X32-bit register file structure

Load/Store Instructions

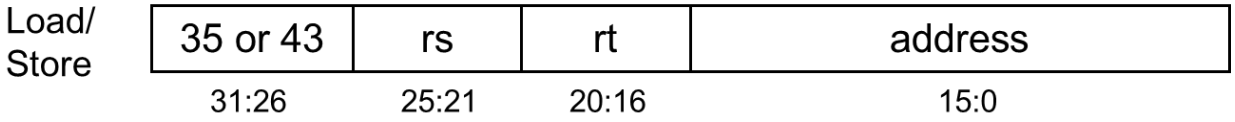
- Read register operands
- Calculate address using 16-bit offset
  - Use ALU, but sign-extend offset
- Load: Read memory and update register
- Store: Write register value to memory



a. Data memory unit



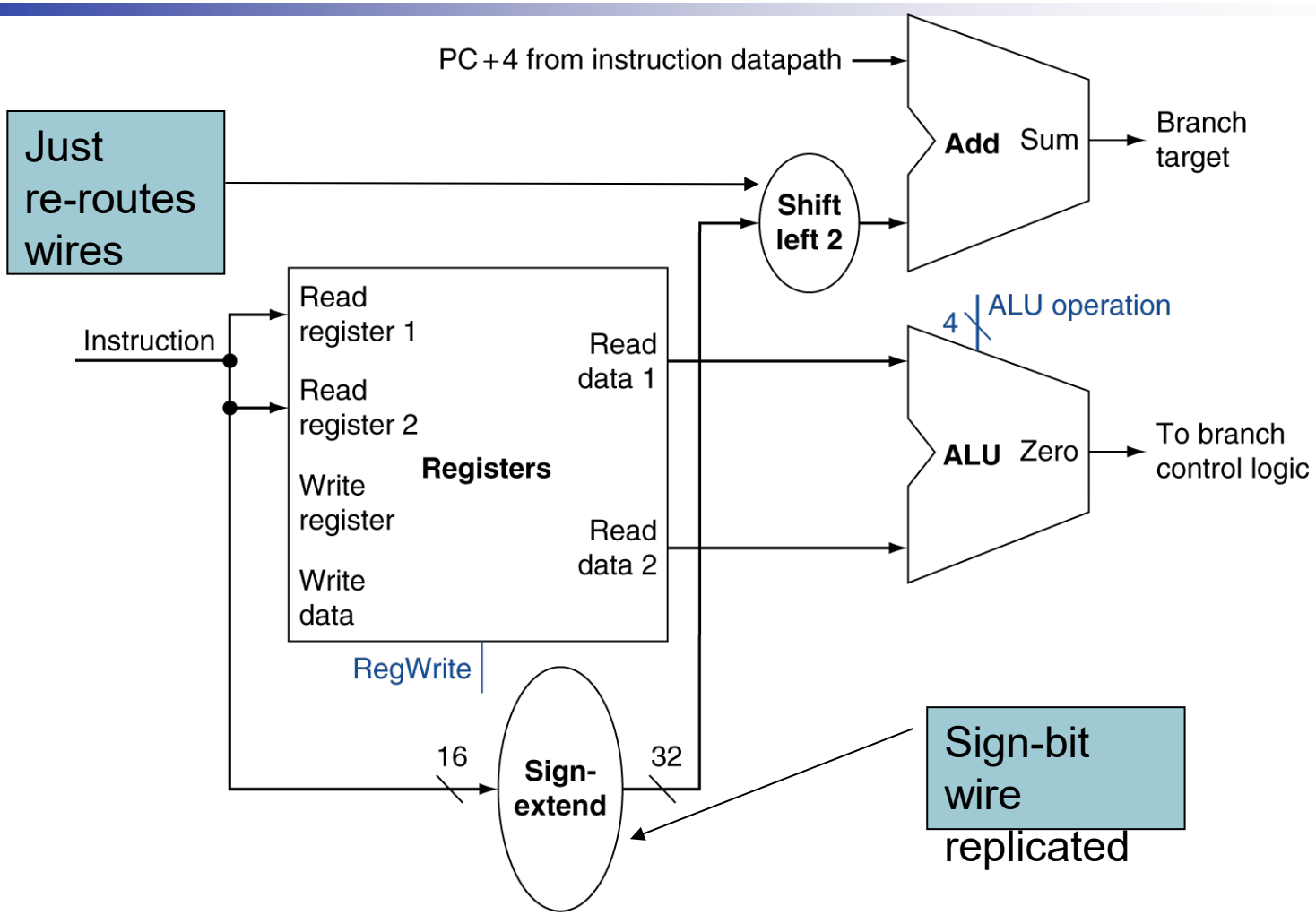
b. Sign extension unit

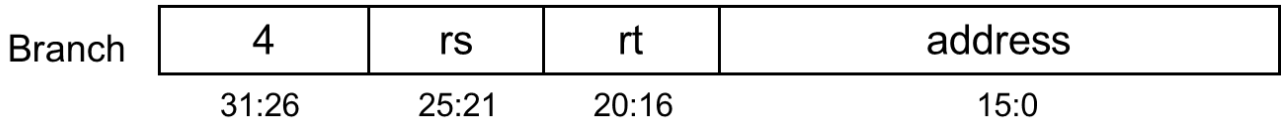


Branch Instructions

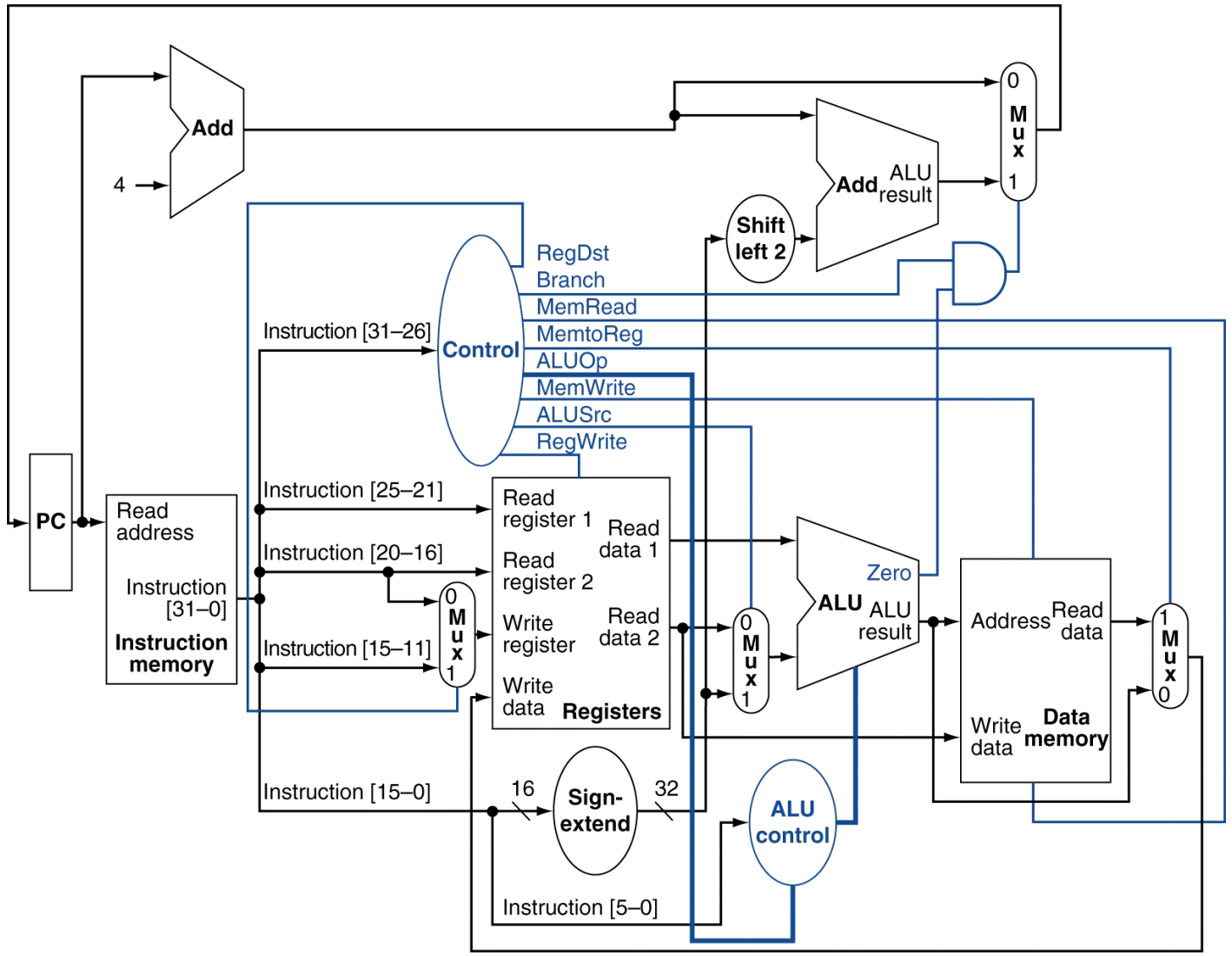
- Read register operands
- Compare operands
  - Use ALU, subtract and check Zero output
- Calculate target address
  - Sign-extend displacement
  - Shift left 2 places (word displacement)
  - Add to PC + 4
    - Already calculated by instruction fetch

Branch Instructions

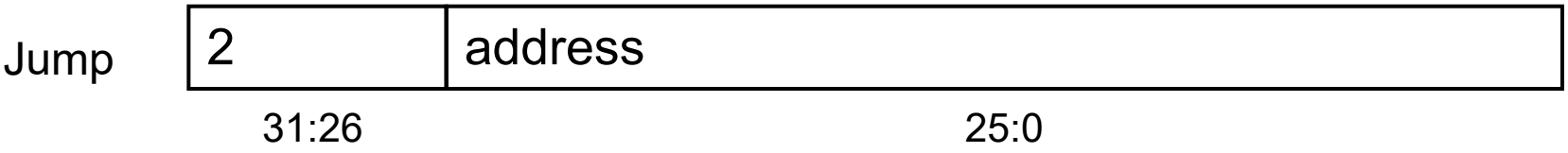




Single-Cycle Processor

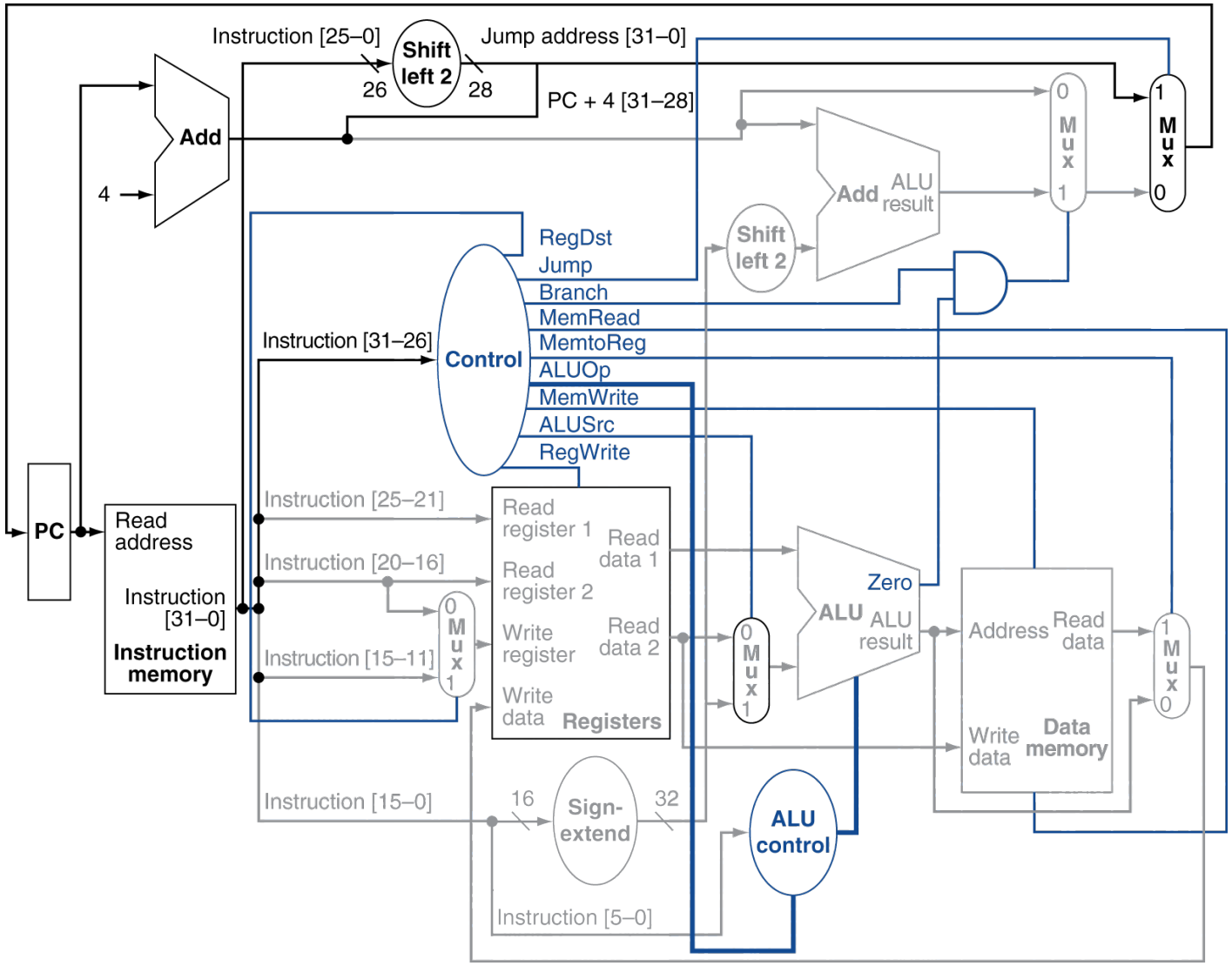


Implementing Jumps



- Jump uses word address
- Update PC with concatenation of
  - Top 4 bits of old PC
  - 26-bit jump address
  - 00
- Need an extra control signal decoded from opcode

Single-Cycle Processor

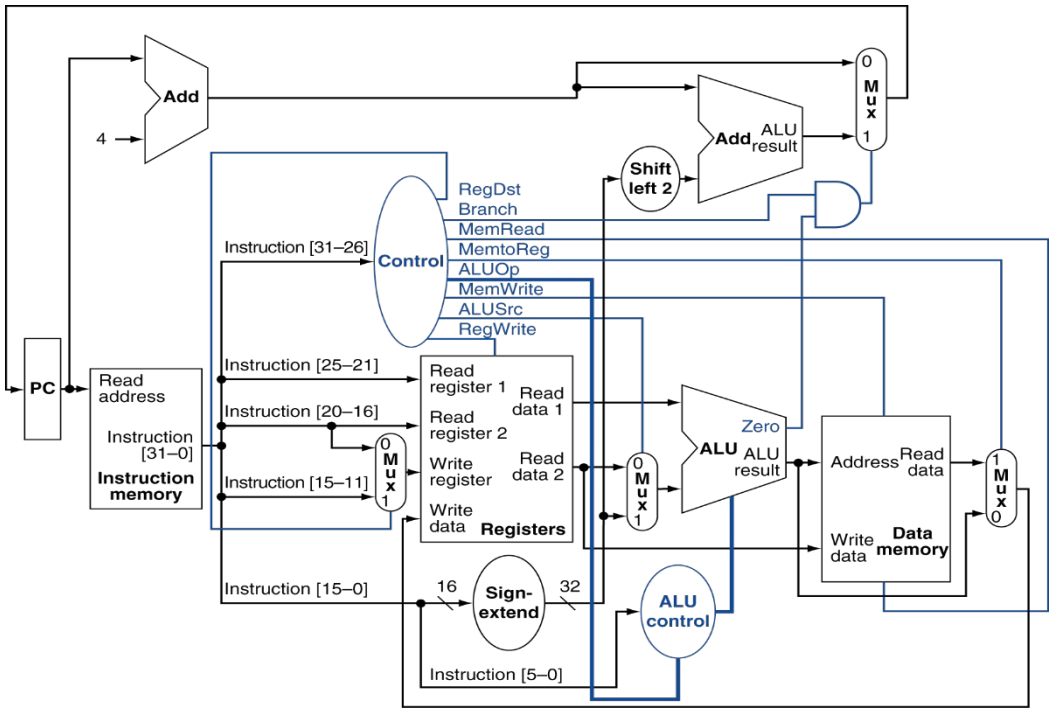


op	jump target

J format

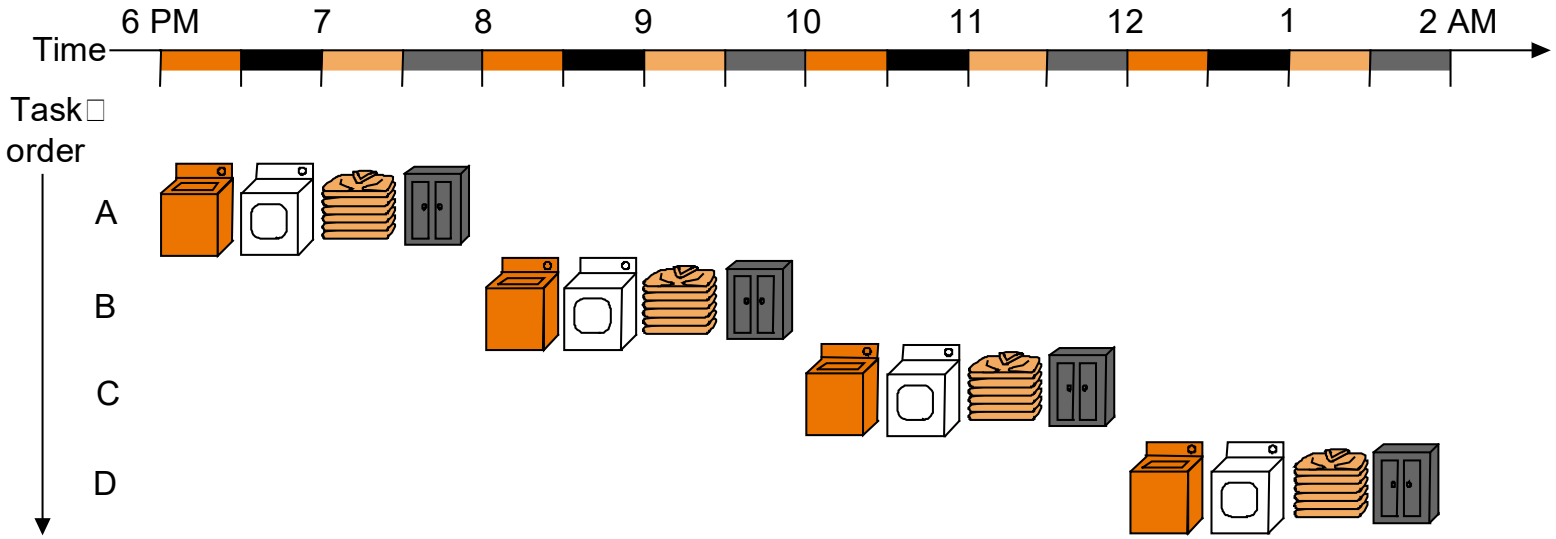
Performance Issues

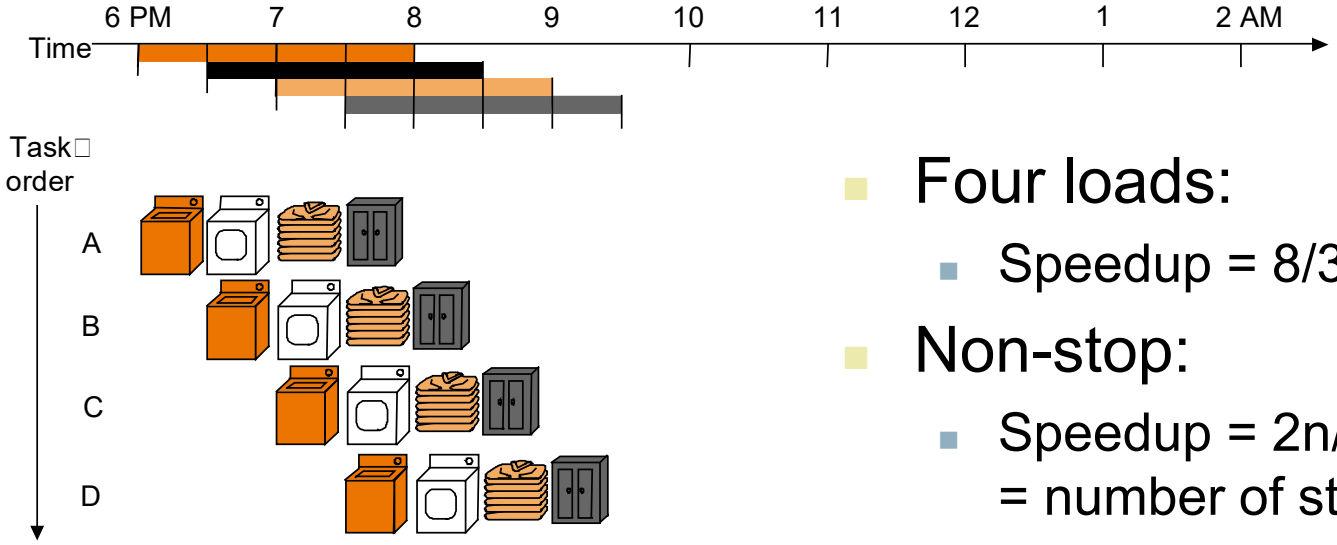
- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file



Pipeline

Pipelining Multiple Loads of Laundry





- Four loads:
  - Speedup = 8/3.5 = 2.3
- Non-stop:
  - Speedup = 2n/(0.5n + 1.5) ≈ 4
    = number of stages

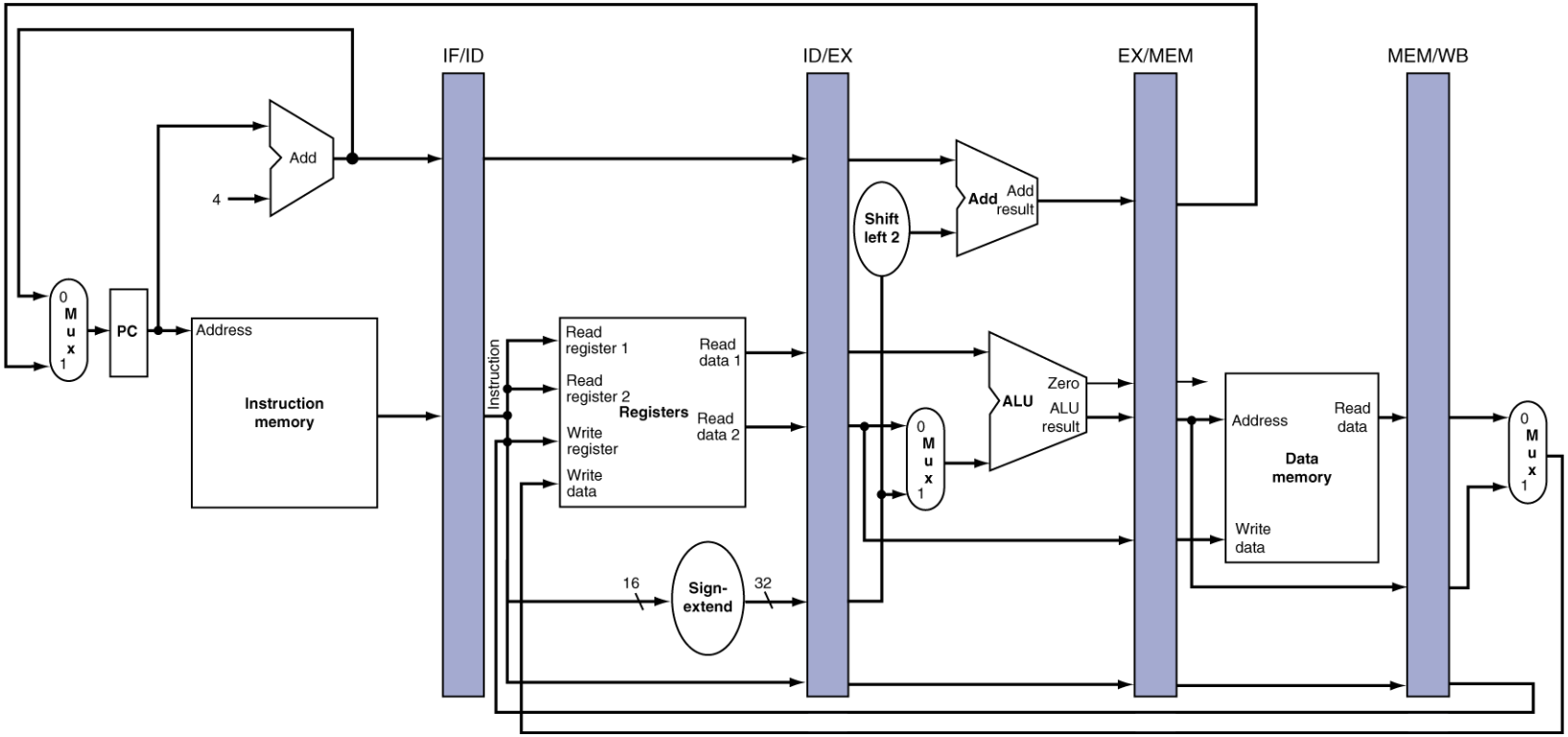
MIPS Pipeline

- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

Single-Cycle Pipeline Diagram

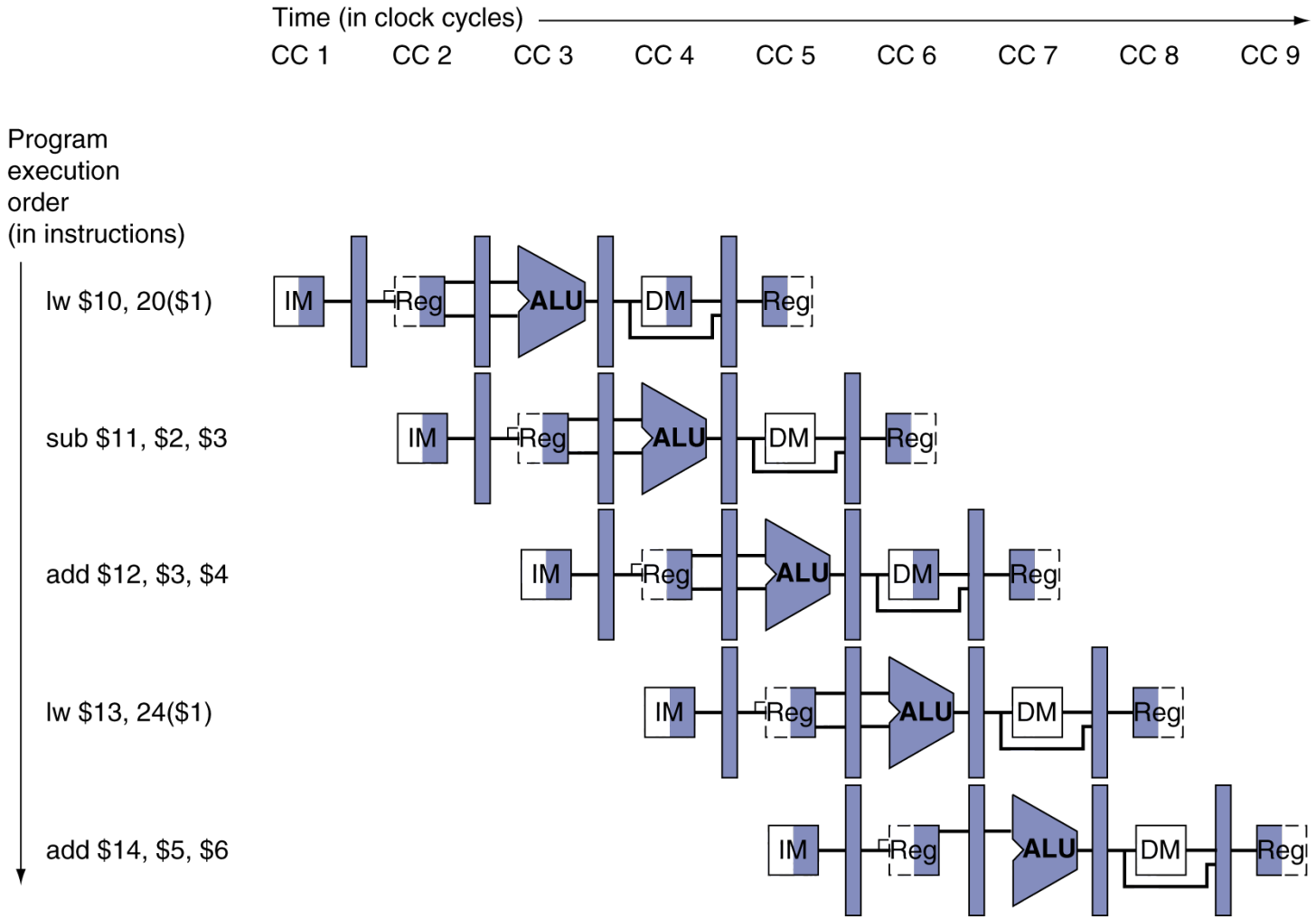
- State of pipeline in a given cycle

add $14, $5, $6	lw $13, 24 ($1)	add $12, $3, $4	sub $11, $2, $3	lw $10, 20($1)
Instruction fetch	Instruction decode	Execution	Memory	Write-back



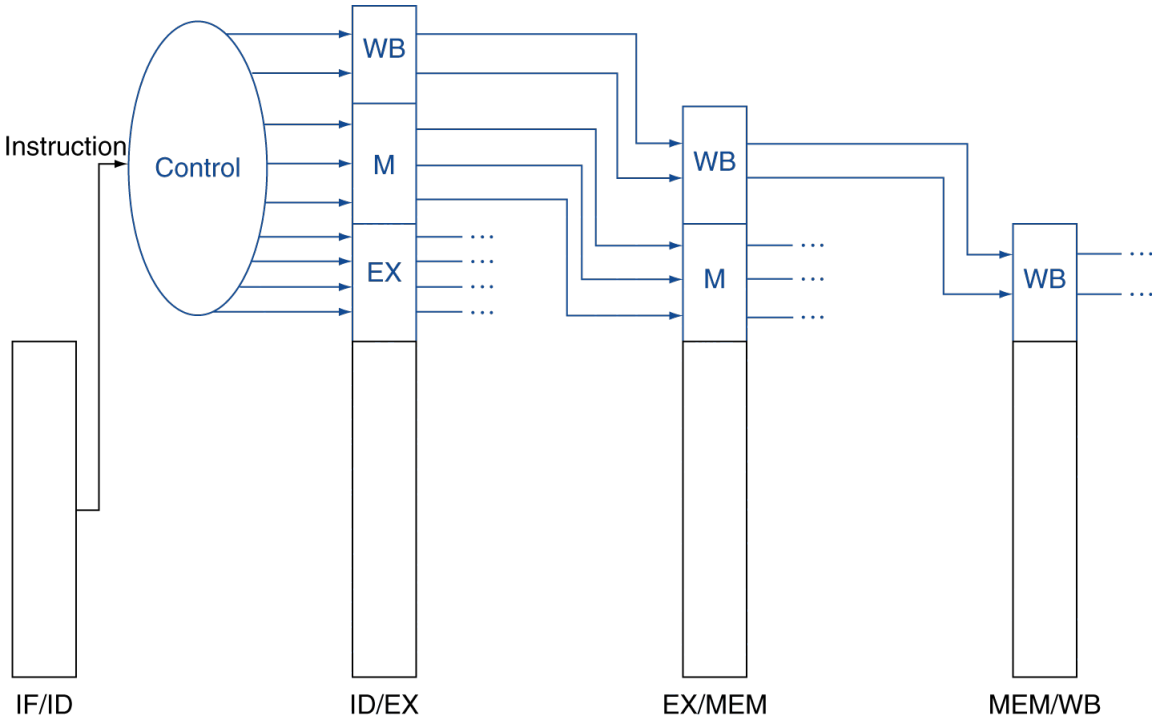
Multi-Cycle Pipeline Diagram

- Form showing resource usage

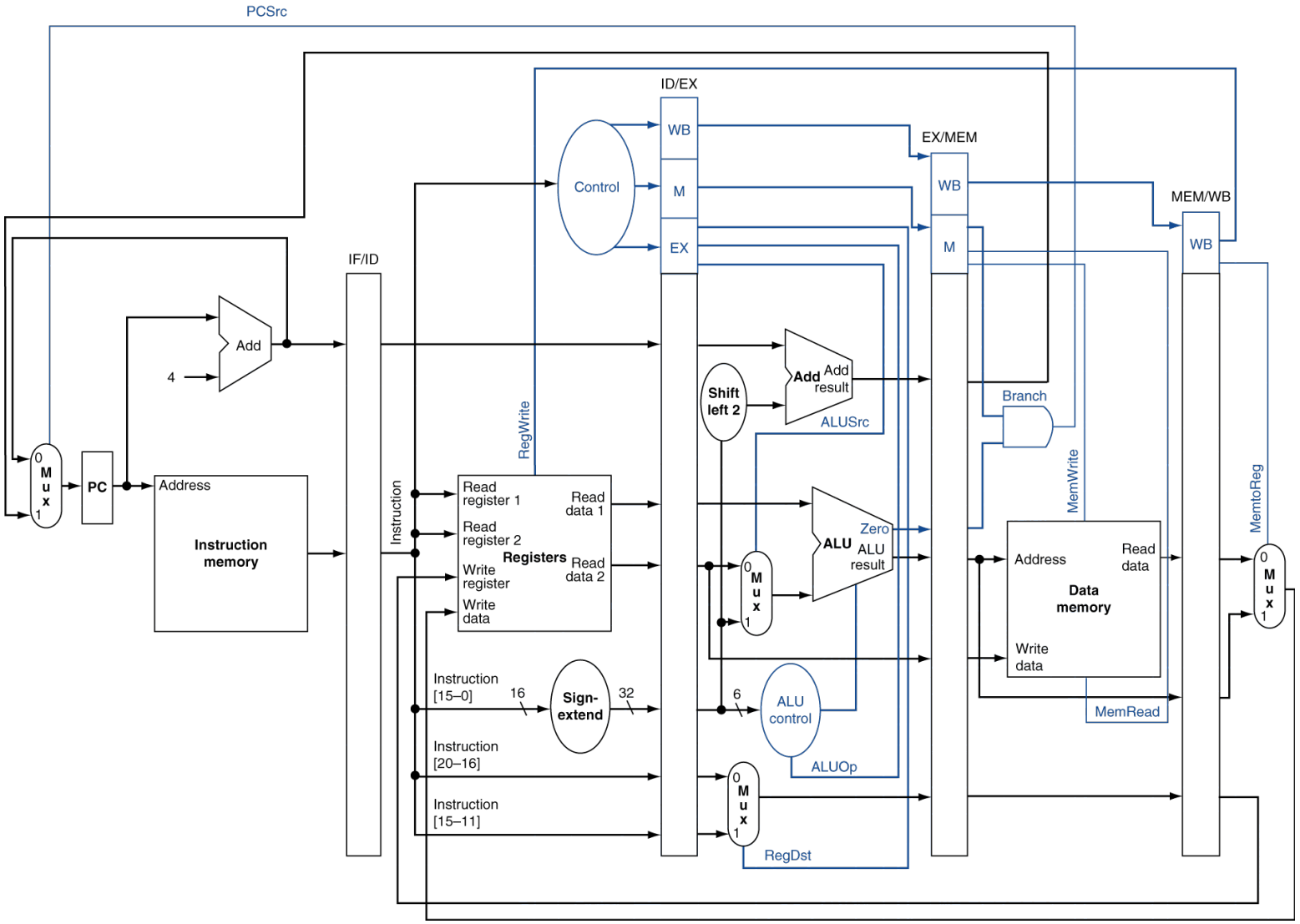


Pipelined Control

- Control signals derived from instruction
  - As in single-cycle implementation



Pipelined Control

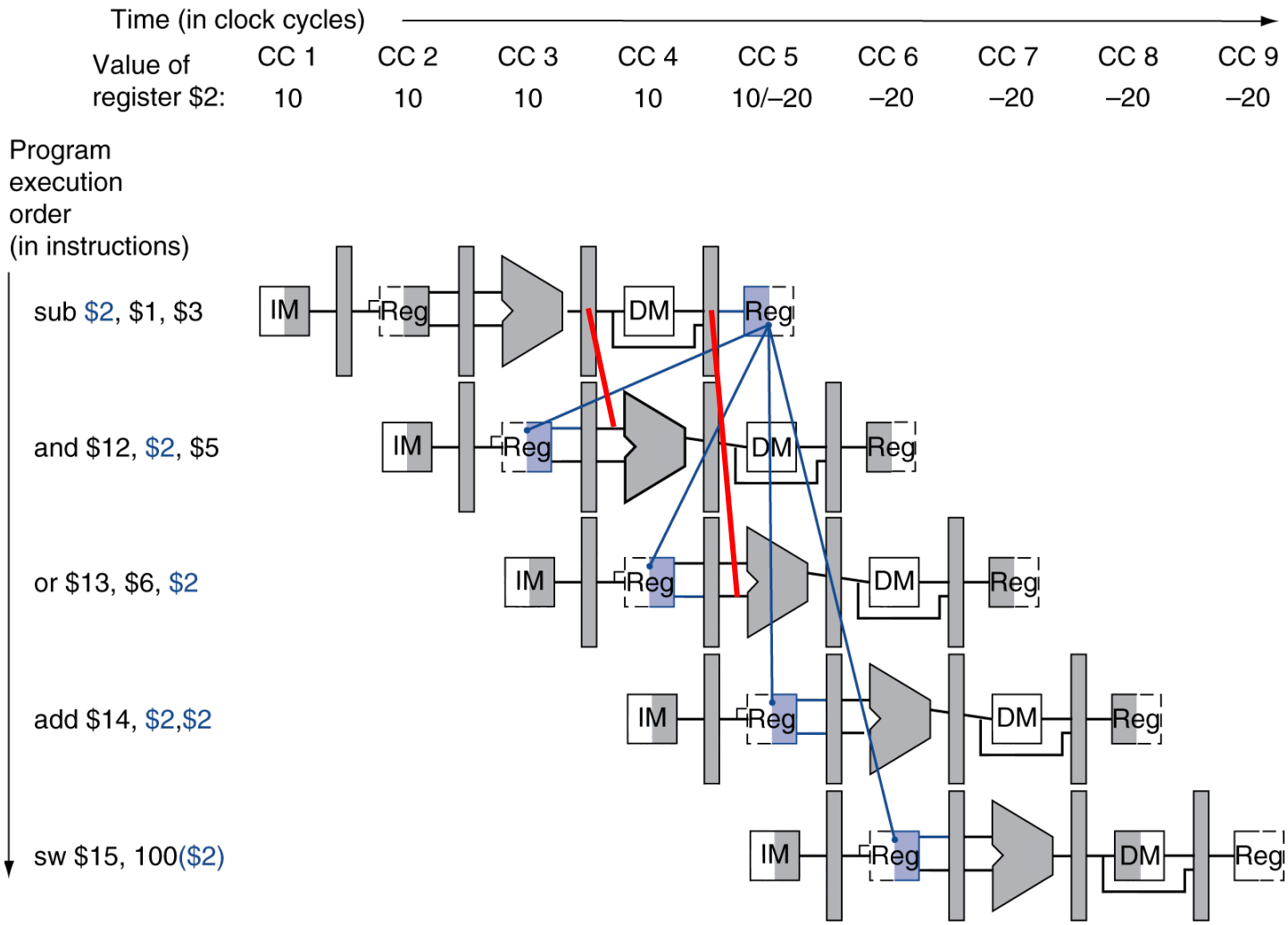


Hazards

Hazards

- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
  - A required resource is busy
- Data hazard
  - Need to wait for previous instruction to complete its data read/write
- Control hazard
  - Deciding on control action depends on previous instruction

Data Hazards & Forwarding



Detecting the Need to Forward

- Pass register numbers along pipeline
  - e.g., ID/EX.RegisterRs = register number for Rs sitting in ID/EX pipeline register
- ALU operand register numbers in EX stage are given by
  - ID/EX.RegisterRs, ID/EX.RegisterRt
- Data hazards when
  - 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs
  - 1b. EX/MEM.RegisterRd = ID/EX.RegisterRt
  - 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs
  - 2b. MEM/WB.RegisterRd = ID/EX.RegisterRt

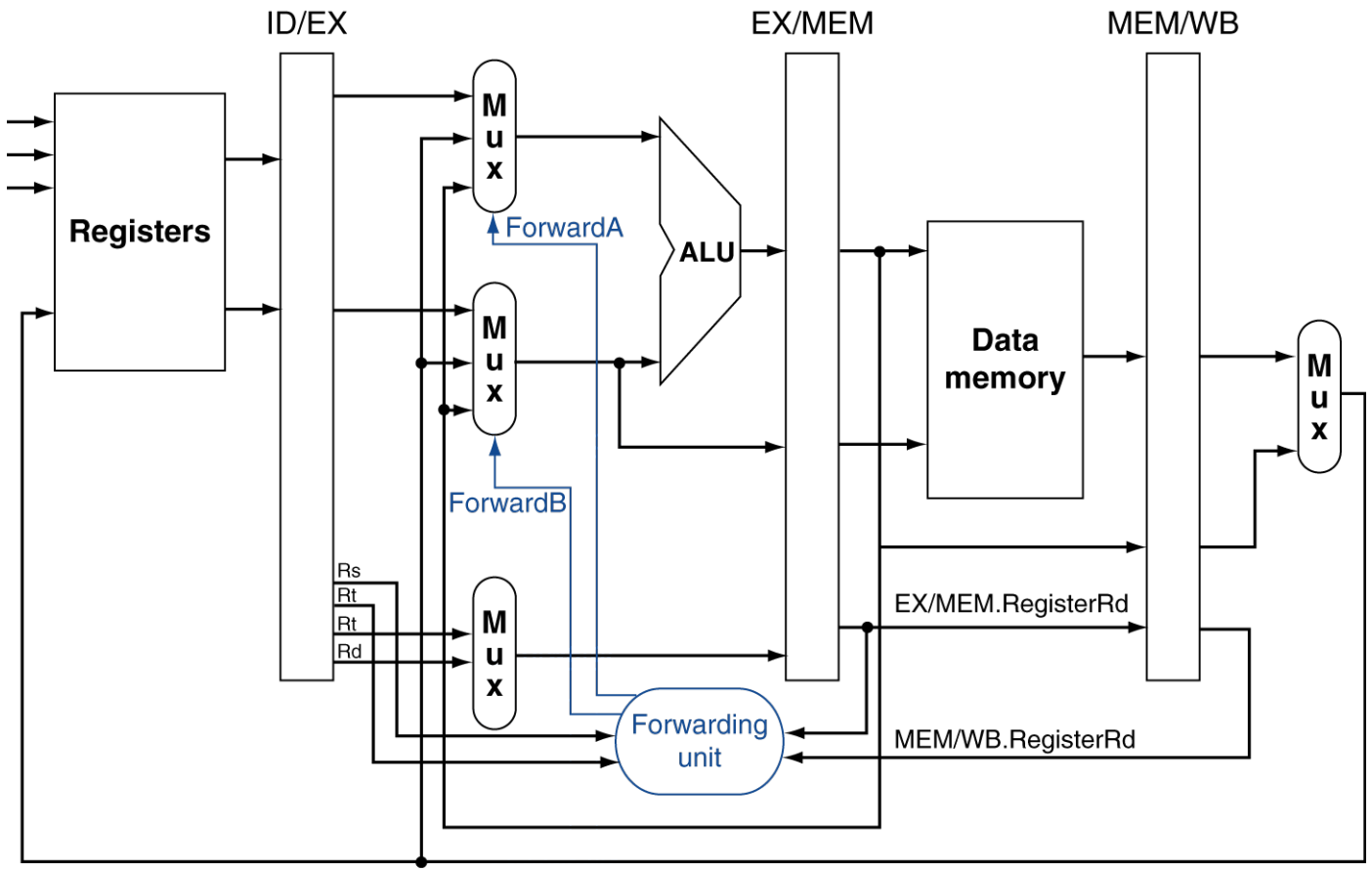
Fwd from EX/MEM pipeline reg

Fwd from MEM/WB pipeline reg

Detecting the Need to Forward

- But only if forwarding instruction will write to a register!
  - EX/MEM.RegWrite, MEM/WB.RegWrite
- And only if Rd for that instruction is not $zero
  - EX/MEM.RegisterRd ≠ 0, MEM/WB.RegisterRd ≠ 0

Forwarding Paths



b. With forwarding

Double Data Hazard

- Consider the sequence:

```
add $1,$1,$2
add $1,$1,$3
add $1,$1,$4
```

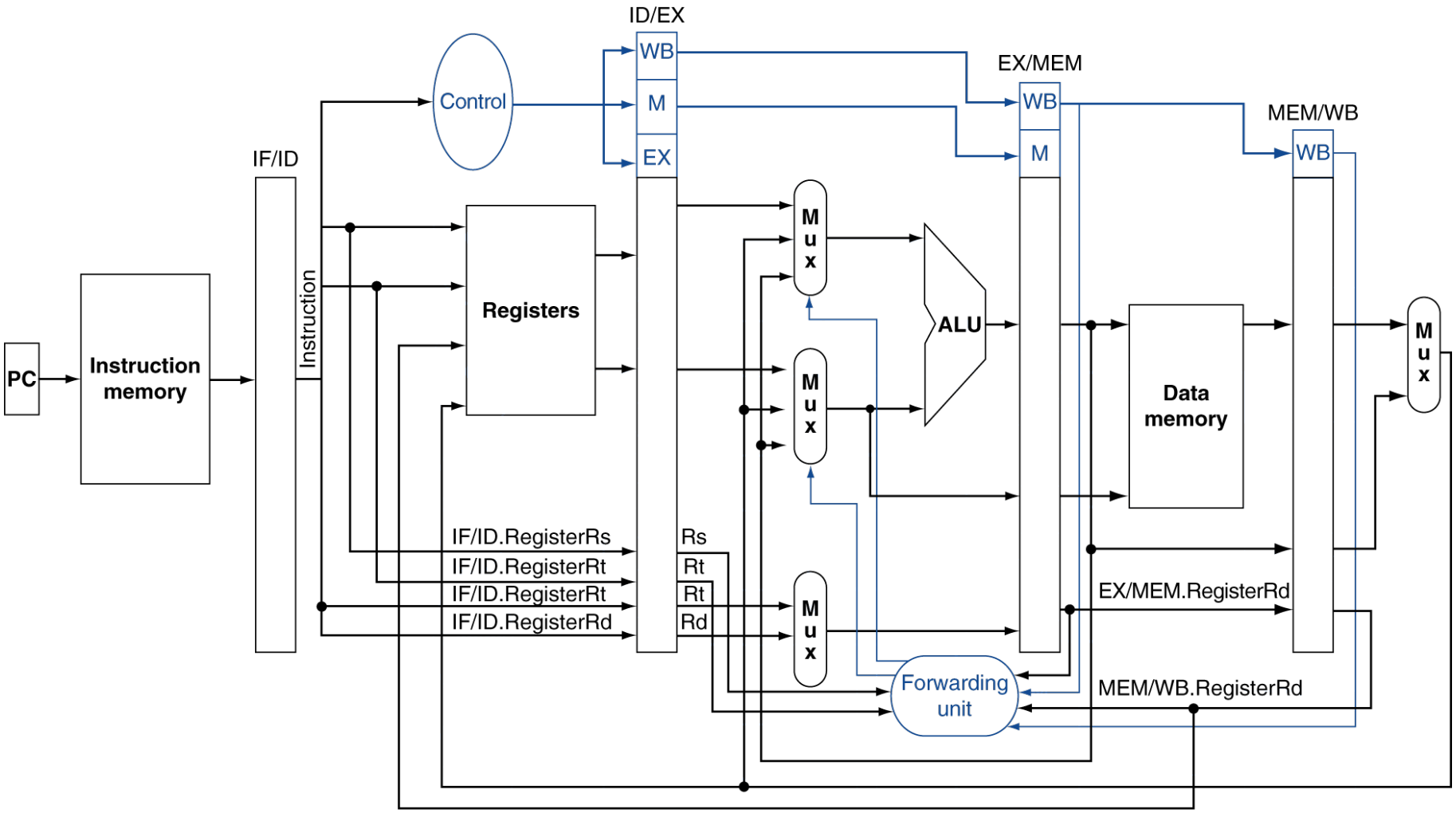
- Both hazards occur
  - Want to use the most recent
- Revise MEM hazard condition
  - Only fwd if EX hazard condition isn't true

Revised Forwarding Condition

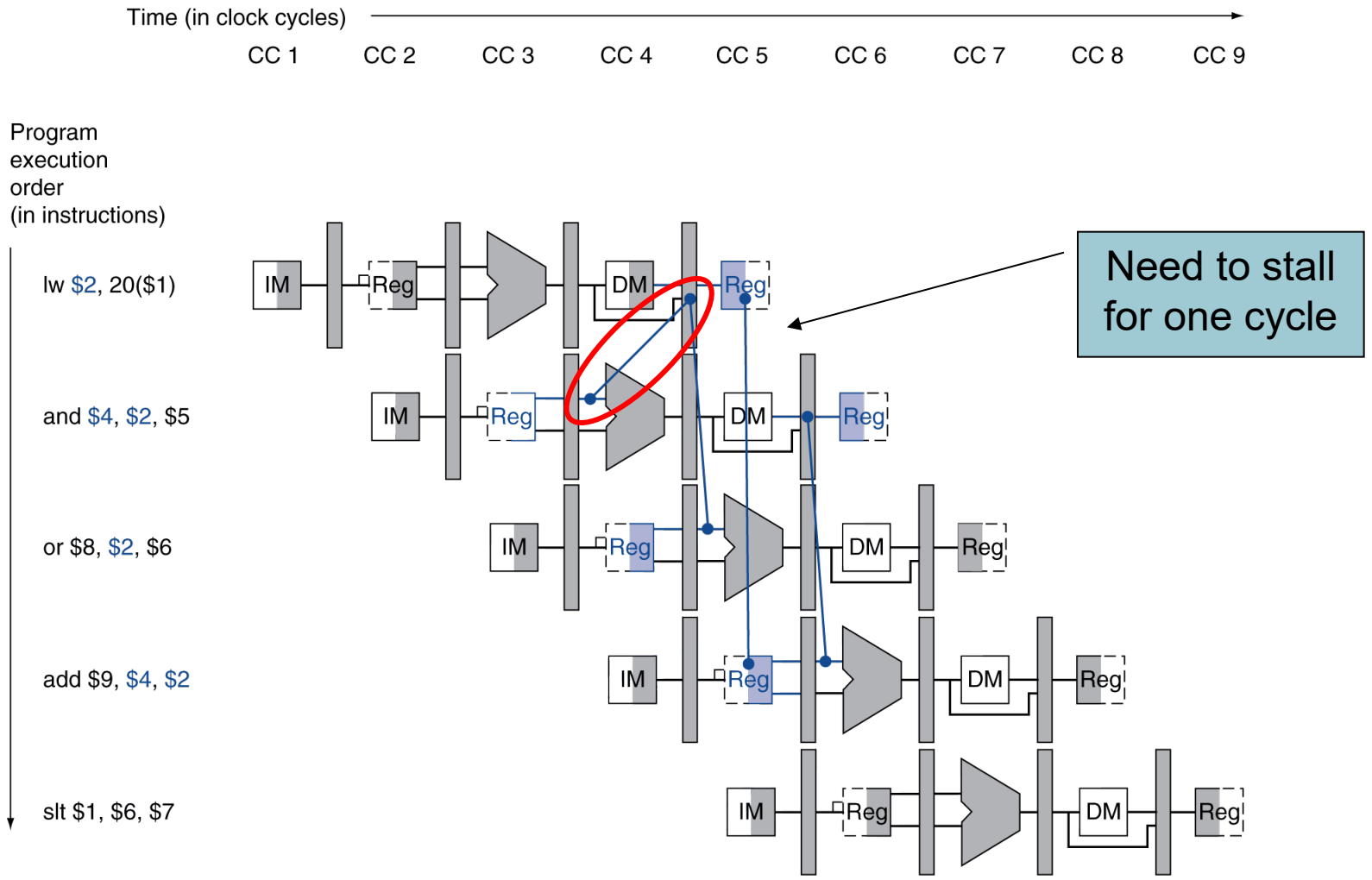
- MEM hazard
  - if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs))
    ForwardA = 01
  - if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRt))
    ForwardB = 01

```
add $1,$1,$2
add $1,$1,$3
add $1,$1,$4
```

Datapath with Forwarding



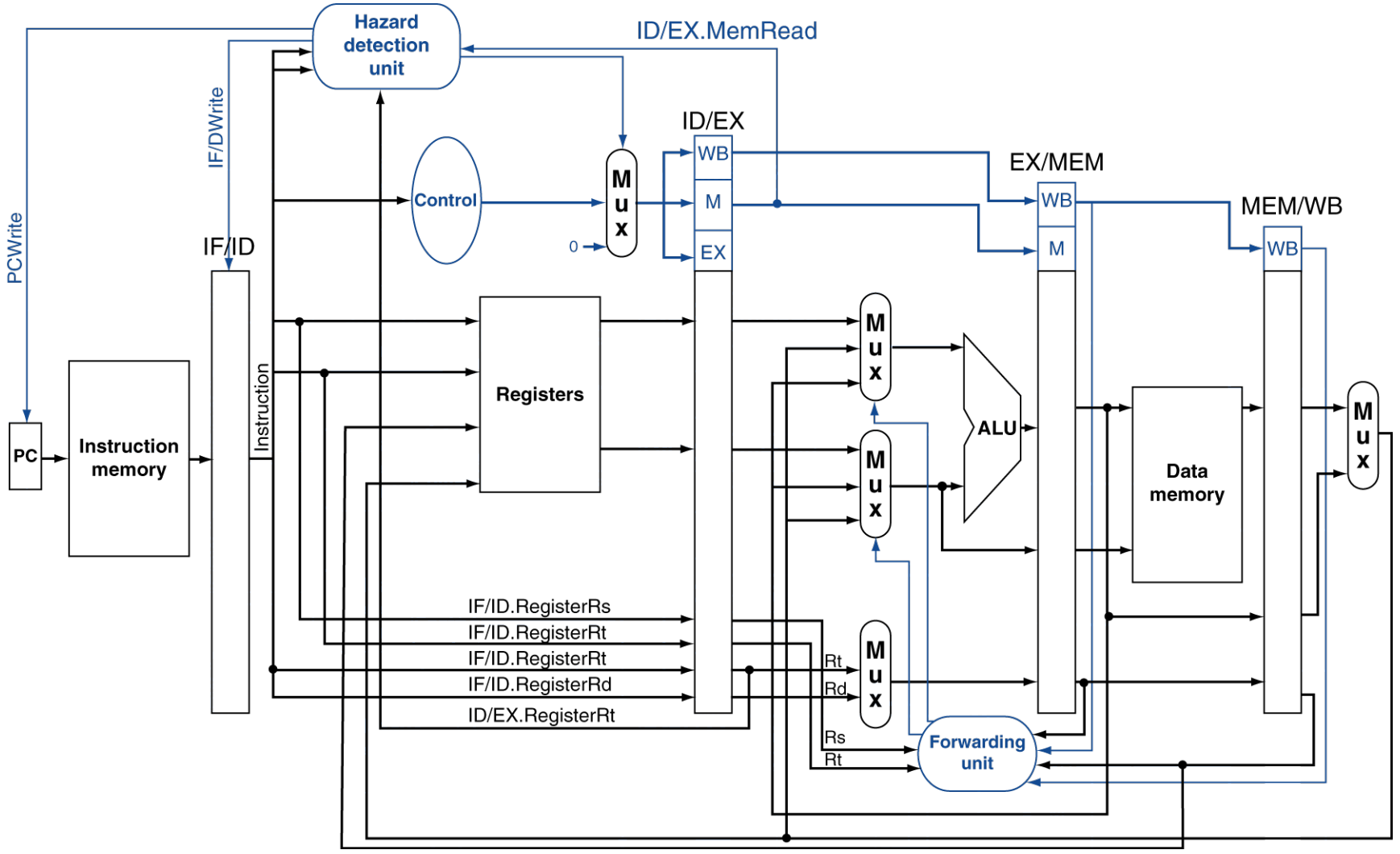
Load-Use Data Hazard



Load-Use Hazard Detection

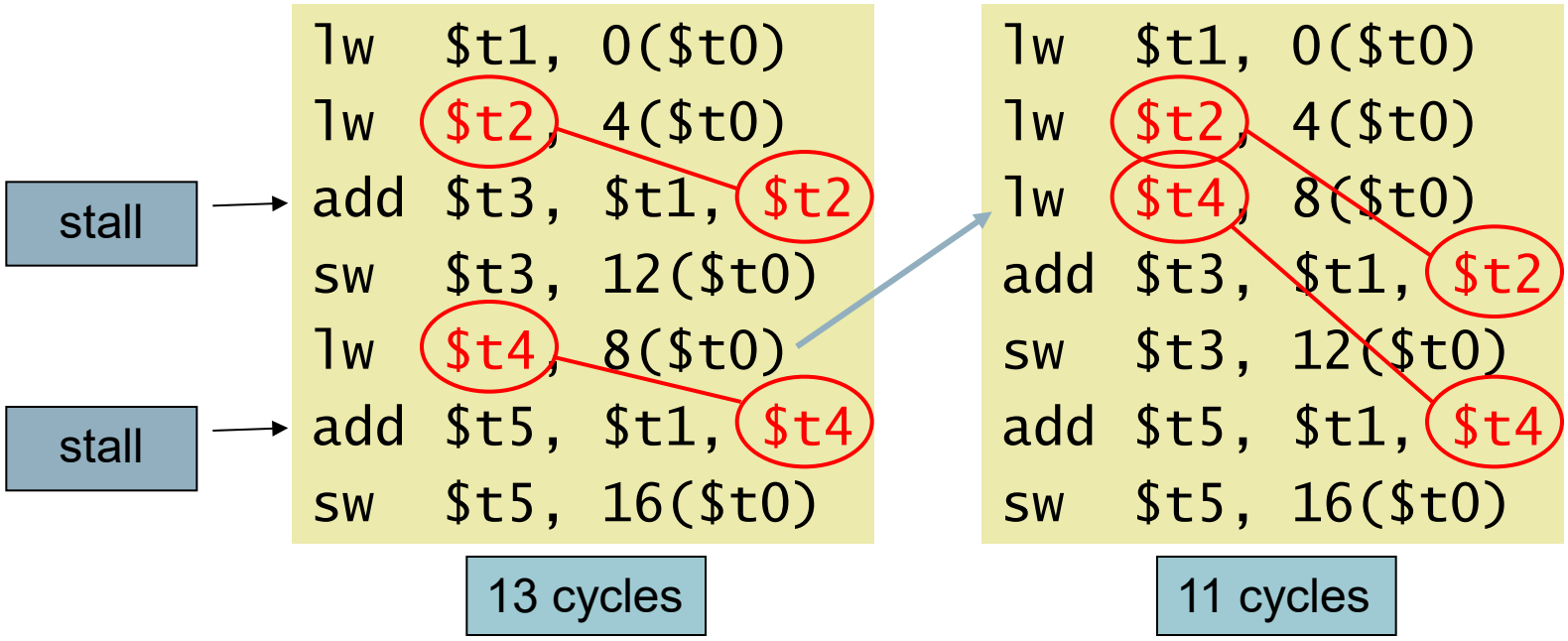
- Check when using instruction is decoded in ID stage
- ALU operand register numbers in ID stage are given by
  - IF/ID.RegisterRs, IF/ID.RegisterRt
- Load-use hazard when
  - ID/EX.MemRead and
    ((ID/EX.RegisterRt = IF/ID.RegisterRs) or
    (ID/EX.RegisterRt = IF/ID.RegisterRt))
- If detected, stall and insert bubble

Datapath with Hazard Detection



Code Scheduling to Avoid Stalls

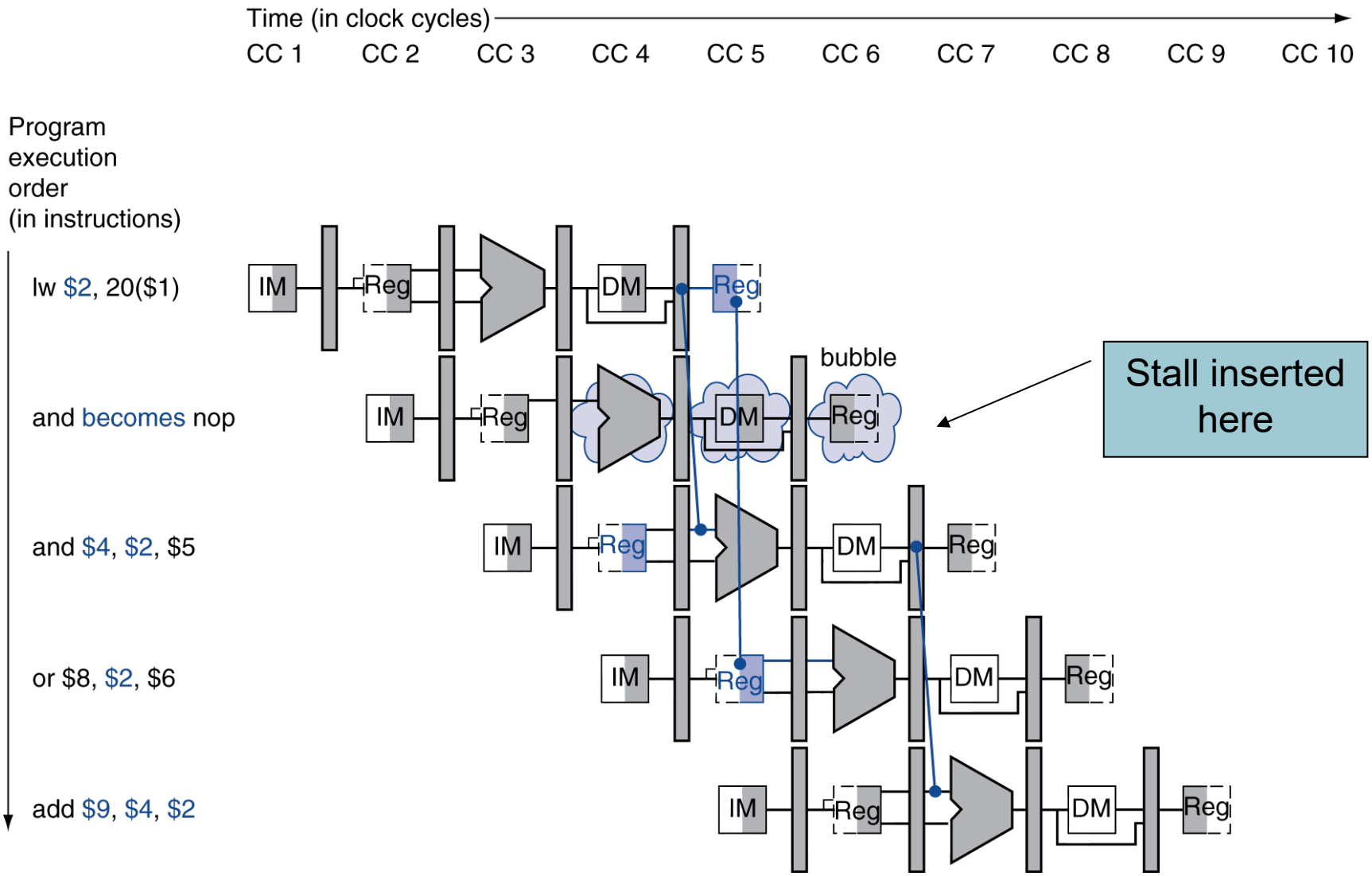
- Reorder code to avoid use of load result in the next instruction
- C code for `A = B + E; C = B + F;`



How to Stall the Pipeline

- Force control values in ID/EX register to 0
  - EX, MEM and WB do nop (no-operation)
- Prevent update of PC and IF/ID register
  - Using instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows MEM to read data for lw
    - Can subsequently forward to EX stage

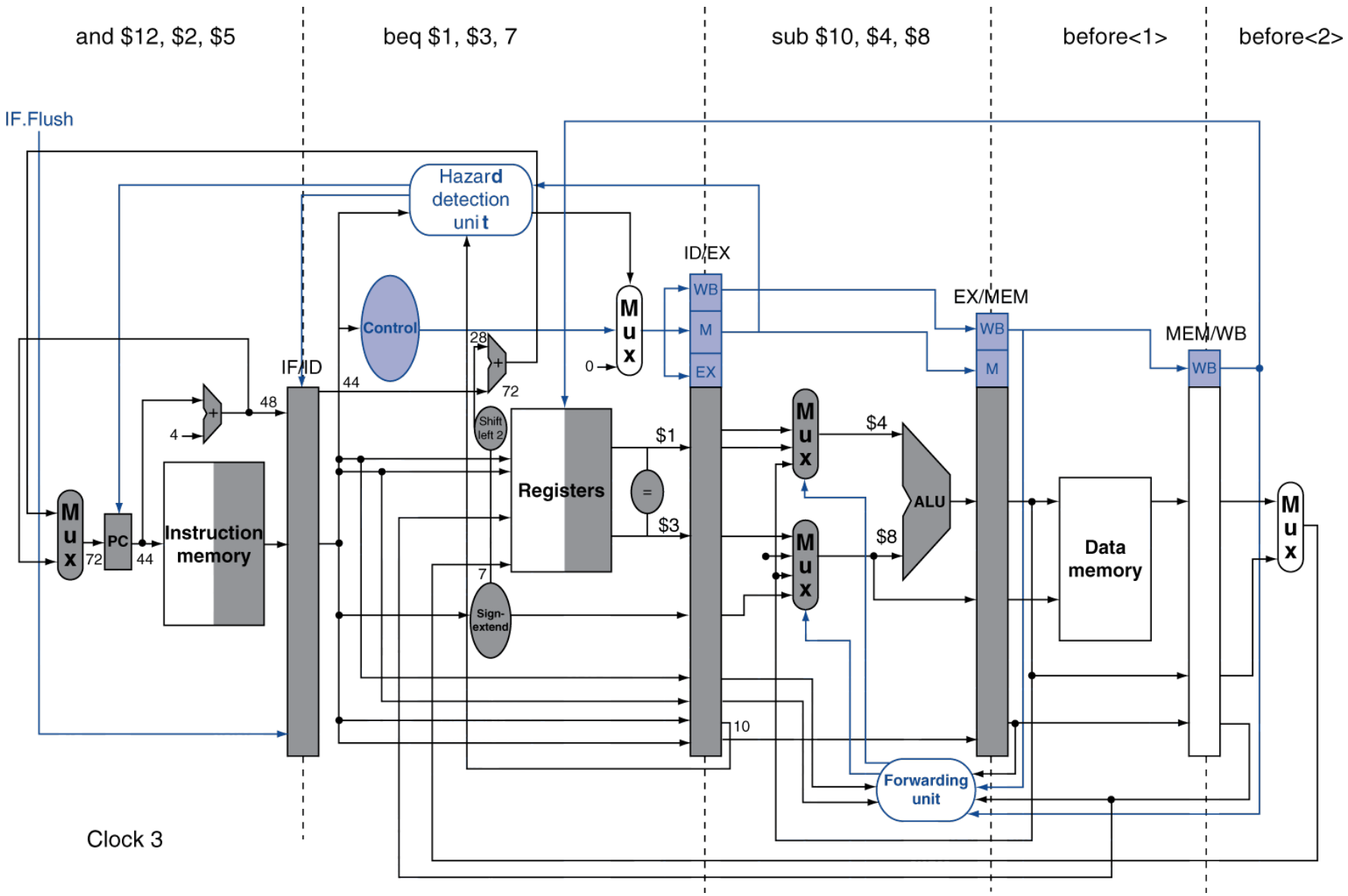
Load-Use Data Hazard



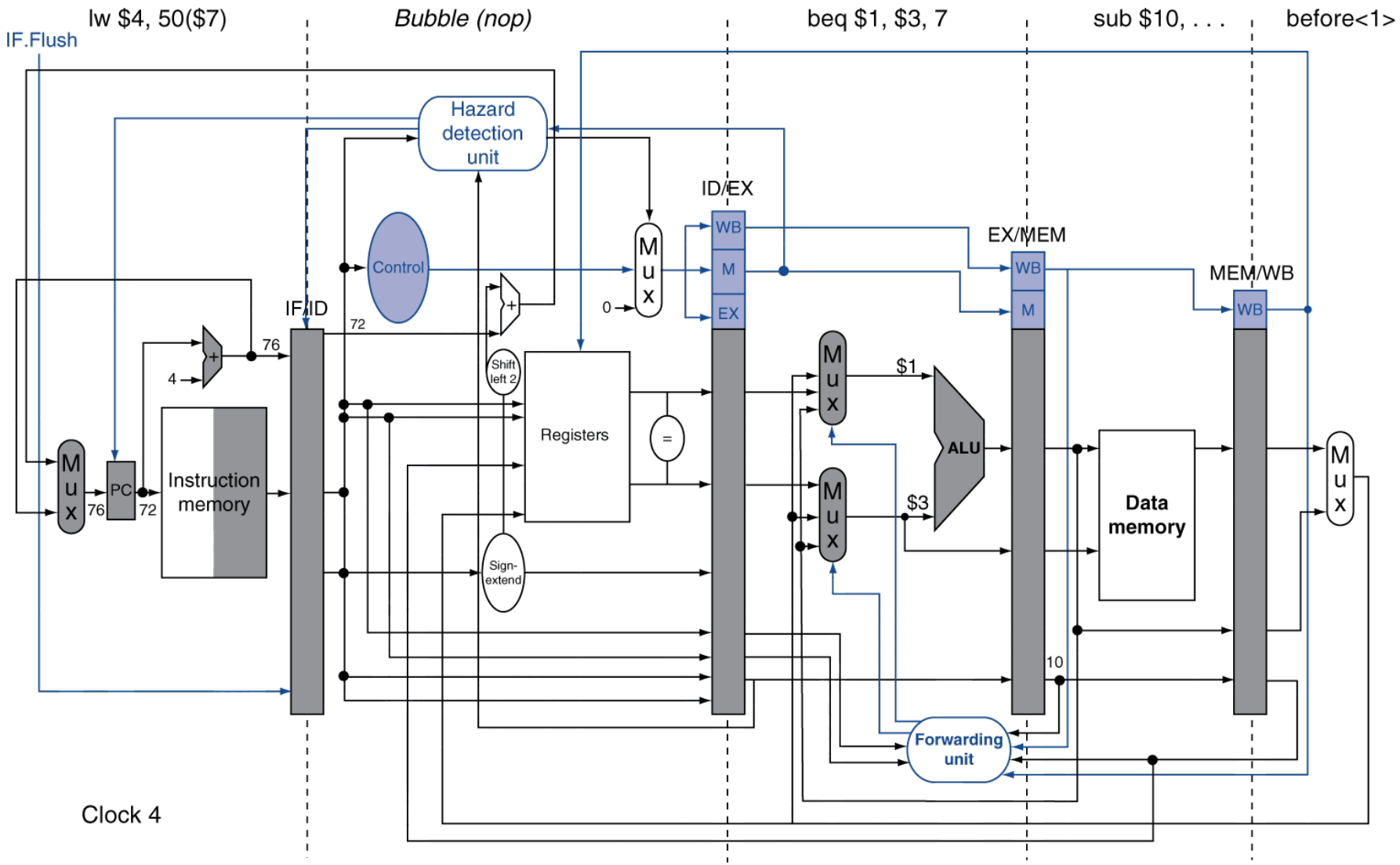
Control Hazards

- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline can't always fetch correct instruction
    - Still working on ID stage of branch
- In MIPS pipeline
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in ID stage

Example: Branch Taken



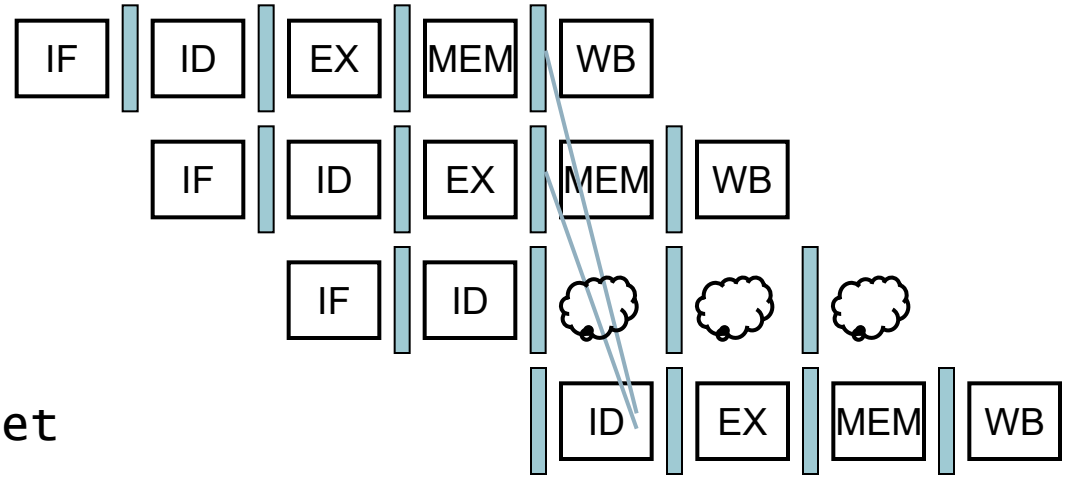
Example: Branch Taken



Data Hazards for Branches

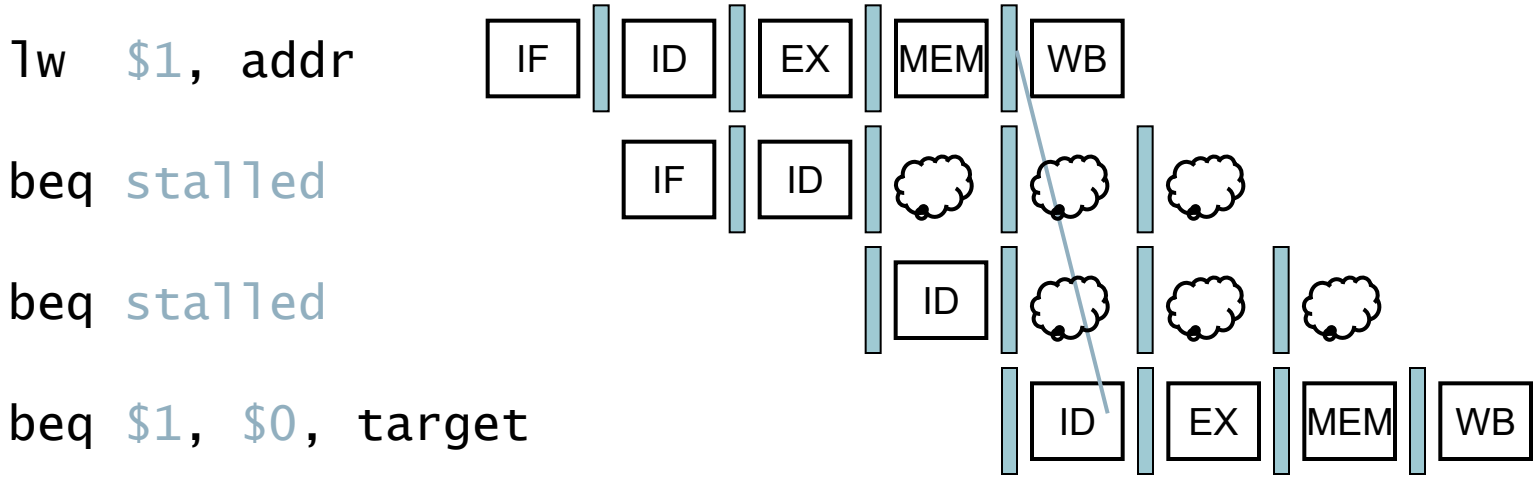
- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction
  - Need 1 stall cycle

```
lw  $1, addr
add $4, $5, $6
beq stalled
beq $1, $4, target
```



Data Hazards for Branches

- If a comparison register is a destination of immediately preceding load instruction
  - Need 2 stall cycles

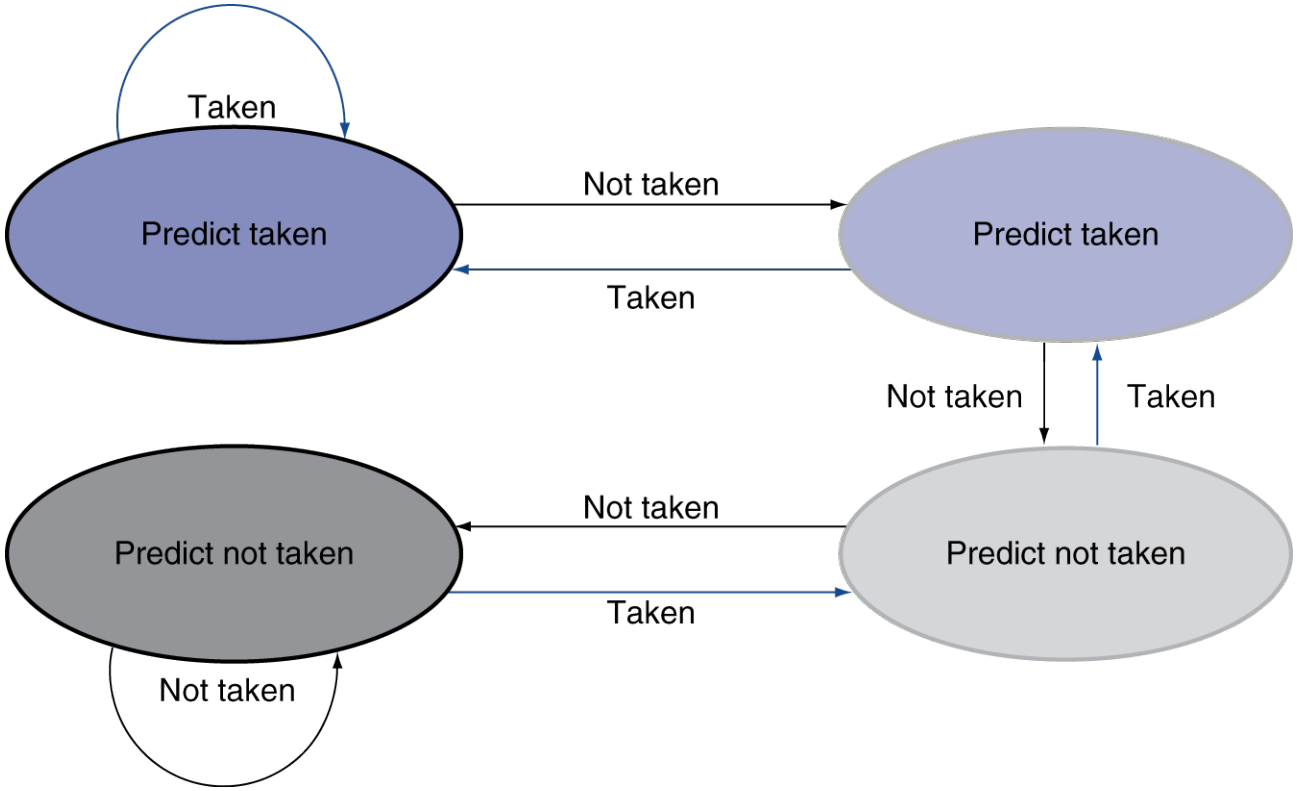


Dynamic Branch Prediction

- In deeper and superscalar pipelines, branch penalty is more significant
- Use dynamic prediction
  - Branch prediction buffer (aka branch history table)
  - Indexed by recent branch instruction addresses
  - Stores outcome (taken/not taken)
  - To execute a branch
    - Check table, expect the same outcome
    - Start fetching from fall-through or target
    - If wrong, flush pipeline and flip prediction

2-Bit Predictor

- Only change prediction on two successive mispredictions



Calculating the Branch Target

- Even with predictor, still need to calculate the target address
  - 1-cycle penalty for a taken branch
- Branch target buffer
  - Cache of target addresses
  - Indexed by PC when instruction fetched
    - If hit and instruction is branch predicted taken, can fetch target immediately

Exceptions

Handling Exceptions

- In MIPS, exceptions managed by a System Control Coprocessor (CP0)
- Save PC of offending (or interrupted) instruction
  - In MIPS: Exception Program Counter (EPC)
- Save indication of the problem
  - In MIPS: Cause register
  - We'll assume 1-bit
    - 0 for undefined opcode, 1 for overflow
- Jump to handler at e.g., 8000 0180

Handler Actions

- Read cause, and transfer to relevant handler
- Determine action required
- If restartable
  - Take corrective action
  - use EPC to return to program
- Otherwise
  - Terminate program
  - Report error using EPC, cause, …

Parallelism via Instructions

Instruction-Level Parallelism (ILP)

- Pipelining: executing multiple instructions in parallel
- To increase ILP
  - Deeper pipeline
    - Less work per stage ⇒ shorter clock cycle
  - Multiple issue
    - Replicate pipeline stages ⇒ multiple pipelines
    - Start multiple instructions per clock cycle
    - CPI < 1, so use Instructions Per Cycle (IPC)
    - E.g., 4GHz 4-way multiple-issue
      - 16 BIPS, peak CPI = 0.25, peak IPC = 4
    - But dependencies reduce this in practice

Multiple Issue

- Static multiple issue
  - Compiler groups instructions to be issued together
  - Packages them into “issue slots”
  - Compiler detects and avoids hazards
- Dynamic multiple issue
  - CPU examines instruction stream and chooses instructions to issue each cycle
  - Compiler can help by reordering instructions
  - CPU resolves hazards using advanced techniques at runtime

Scheduling Static Multiple Issue

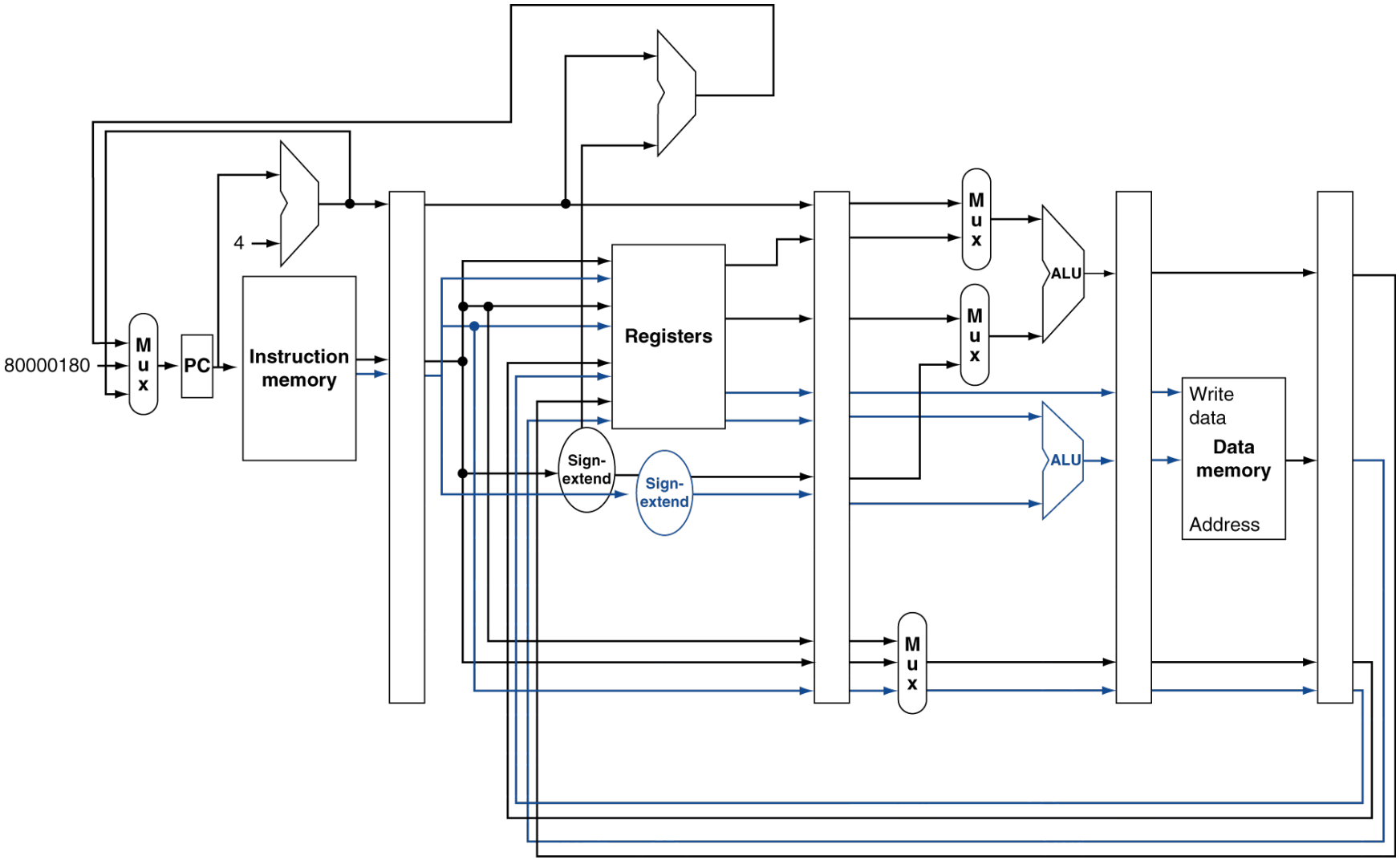
- Compiler must remove some/all hazards
  - Reorder instructions into issue packets
  - No dependencies with a packet
  - Possibly some dependencies between packets
    - Varies between ISAs; compiler must know!
  - Pad with nop if necessary

MIPS with Static Dual Issue

- Two-issue packets
  - One ALU/branch instruction
  - One load/store instruction
  - 64-bit aligned
    - ALU/branch, then load/store
    - Pad an unused instruction with nop

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

MIPS with Static Dual Issue



Hazards in the Dual-Issue MIPS

- More instructions executing in parallel
- EX data hazard
  - Forwarding avoided stalls with single-issue
  - Now can't use ALU result in load/store in same packet
    - ```
      add  $t0, $s0, $s1
      load $s2, 0($t0)
      ```
    - Split into two packets, effectively a stall
- Load-use hazard
  - Still one cycle use latency, but now two instructions
- More aggressive scheduling required
      ```

Scheduling Example

- Schedule this for dual-issue MIPS

```
Loop: lw   $t0, 0($s1)       # $t0=array element
      addu $t0, $t0, $s2     # add scalar in $s2
      sw   $t0, 0($s1)       # store result
      addi $s1, $s1,-4       # decrement pointer
      bne  $s1, $zero, Loop  # branch $s1!=0
```

for(i = n; i != 0; i -= 1)

A[i] = A[i] + x;

	ALU/branch	Load/store	cycle
Loop:	nop	lw $t0, 0($s1)	1
	addi $s1, $s1,-4	nop	2
	addu $t0, $t0, $s2	nop	3
	bne $s1, $zero, Loop	sw $t0, 4($s1)	4

- IPC = 5/4 = 1.25 (c.f. peak IPC = 2)

Loop Unrolling

- Replicate loop body to expose more parallelism
  - Reduces loop-control overhead
- Use different registers per replication
  - Called “register renaming”
  - Avoid loop-carried “anti-dependencies”
    - Store followed by a load of the same register
    - Aka “name dependence”
      - Reuse of a register name

Loop Unrolling Example

```
for(i = n; i != 0; i -= 1)
    A[i] = A[i] + x;
```

```
Loop: lw   $t0, 0($s1)
      addu $t0, $t0, $s2
      sw   $t0, 0($s1)
      addi $s1, $s1,-4
       bne  $s1, $zero, Loop
```

```
for(i = n; i != 0; i -= 4)
    A[i] = A[i] + x;
    A[i-1] = A[i-1] + x;
    A[i-2] = A[i-2] + x;
    A[i-3] = A[i-3] + x;
```

```
Loop:  lw
       lw
       lw
       lw
       addu
       addu
       addu
       addu
       sw
       sw
       sw
       sw
       addi $s1, $s1,-16
       bne  $s1, $zero, Loop
```

Loop Unrolling Example

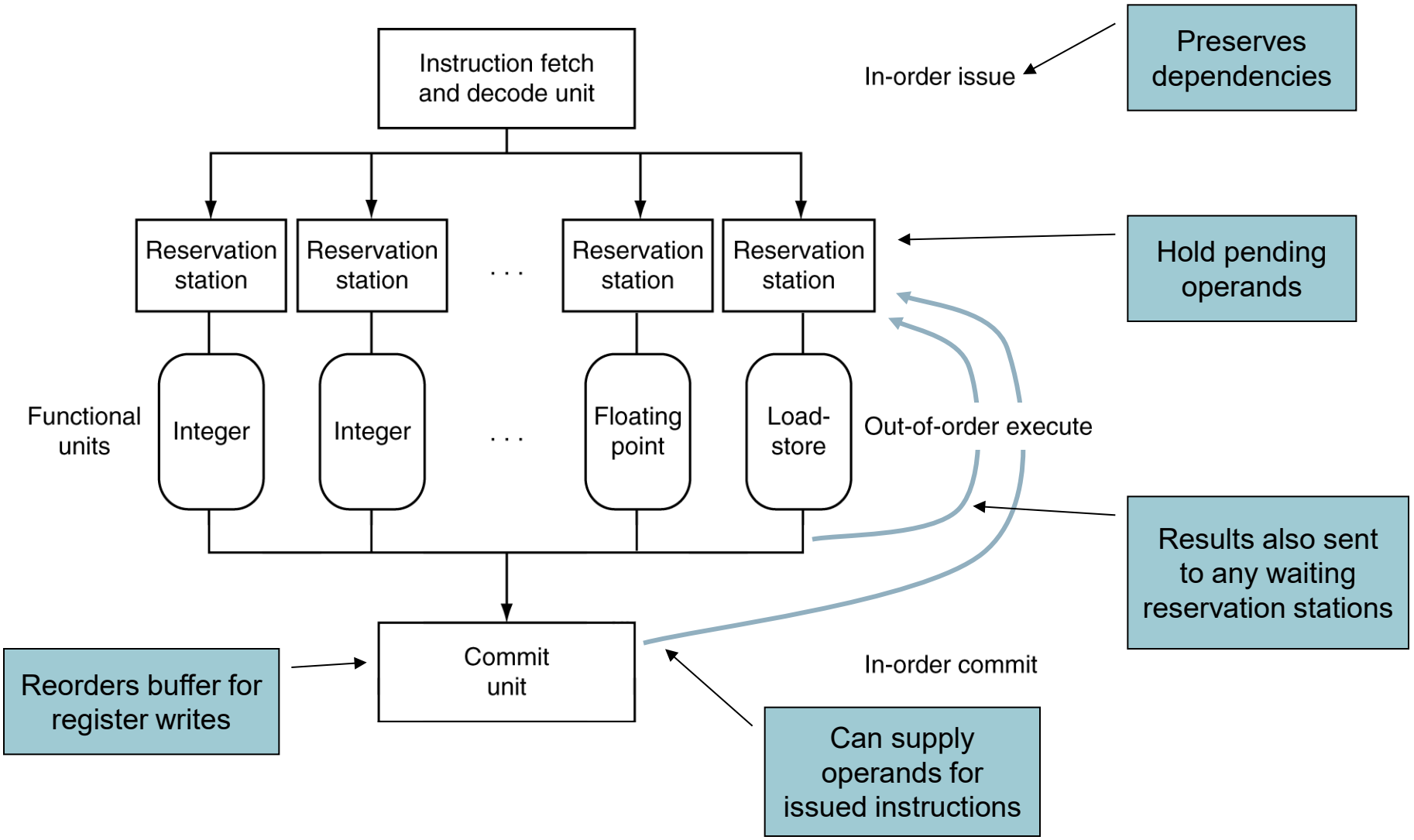
	ALU/branch	Load/store	cycle
Loop:	addi $s1, $s1,-16	lw $t0, 0($s1)	1
	nop	lw $t1, 12($s1)	2
	addu $t0, $t0, $s2	lw $t2, 8($s1)	3
	addu $t1, $t1, $s2	lw $t3, 4($s1)	4
	addu $t2, $t2, $s2	sw $t0, 16($s1)	5
	addu $t3, $t3, $s2	sw $t1, 12($s1)	6
	nop	sw $t2, 8($s1)	7
	bne $s1, $zero, Loop	sw $t3, 4($s1)	8

- IPC = 14/8 = 1.75
  - Closer to 2, but at cost of registers and code size

Dynamic Multiple Issue

- "Superscalar" processors
- CPU decides whether to issue 0, 1, 2, … each cycle
  - Avoiding structural and data hazards
- Avoids the need for compiler scheduling
  - Though it may still help
  - Code semantics ensured by the CPU

Dynamically Scheduled CPU



Register Renaming

- Reservation stations and reorder buffer effectively provide register renaming
- On instruction issue to reservation station
  - If operand is available in register file or reorder buffer
    - Copied to reservation station
    - No longer required in the register; can be overwritten
  - If operand is not yet available
    - It will be provided to the reservation station by a function unit
    - Register update may not be required

Speculation

- "Guess" what to do with an instruction
  - Start operation as soon as possible
  - Check whether guess was right
    - If so, complete the operation
    - If not, roll-back and do the right thing
- Common to static and dynamic multiple issue
- Examples
  - Speculate on branch outcome
    - Roll back if path taken is different
  - Speculate on load
    - Roll back if location is updated

Compiler/Hardware Speculation

- Compiler can reorder instructions
  - e.g., move load before branch
  - Can include “fix-up” instructions to recover from incorrect guess
- Hardware can look ahead for instructions to execute
  - Buffer results until it determines they are actually needed
  - Flush buffers on incorrect speculation

Speculation

- Predict branch and continue issuing
  - Don’t commit until branch outcome determined
- Load speculation
  - Avoid load and cache miss delay
    - Predict the effective address
    - Predict loaded value
    - Load before completing outstanding stores
    - Bypass stored values to load unit
  - Don’t commit load until speculation cleared

Why Do Dynamic Scheduling?

- Why not just let the compiler schedule code?
- Not all stalls are predicable
  - e.g., cache misses
- Can't always schedule around branches
  - Branch outcome is dynamically determined
- Different implementations of an ISA have different latencies and hazards